

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN
CƠ SỞ HỆ THỐNG THÔNG TIN

**CHỦ ĐỀ 30: Hiểu biểu đồ hiệu quả với Visual Token Merging và
Program-of-Thought Learning**

Giảng viên: PGS.TS Hà Quang Thụy

Mã lớp học: INT3201 1

Lớp: K68I-IS

Nhóm: 14

Chu Anh Trường – 23020577

Phạm Quang Vinh – 23020580

MỤC LỤC

Danh sách hình ảnh	4
Danh sách bảng	5
Danh sách thuật ngữ	6
TÓM TẮT DỰ ÁN	7
CHƯƠNG 1. GIỚI THIỆU	8
1.1. Giới thiệu bài báo.....	8
1.2. Mô tả bài toán: Hiểu Biểu Đồ (Chart Understanding).....	8
1.3. Thực trạng, khó khăn và thách thức.....	8
1.4. Bối cảnh và động lực nghiên cứu.....	9
1.5. Giới thiệu tác giả và tổ chức	10
CHƯƠNG 2. KIẾN THỨC NỀN TẢNG	11
2.1. Mô hình Ngôn ngữ lớn Đa phương thức (MLLM).....	11
2.1.1. Kiến trúc chung (Encoder, Connector, LLM)	11
2.1.2. Huấn luyện MLLM	12
2.2. Các phương pháp Hiểu Biểu Đồ	12
2.2.1. Phương pháp dựa trên Pipeline	12
2.2.2. Phương pháp End-to-End.....	13
2.3. Thách thức trong việc Xử lý hình ảnh độ phân giải cao trong MLLM.....	14
2.4. Phương pháp Program-of-Thoughts (PoT)	15
CHƯƠNG 3. PHÂN TÍCH NỘI DUNG BÀI BÁO (TINYCHART)	16
3.1. Chủ đề và Mục tiêu của Bài báo	16
3.2. Các Đóng góp Kỹ thuật chính.....	16
3.3. Phân tích Kiến trúc và Phương pháp luận.....	17
3.3.1. Tổng quan Kiến trúc và Giải pháp VTM	17
3.3.2. Phương pháp học PoT và Bộ dữ liệu ChartQA-PoT.....	18
3.4. Cấu trúc Bài báo.....	19

3.5.	Phân tích Thực nghiệm và Kết quả.....	20
3.5.1.	Thiết lập Thực nghiệm.....	20
3.5.2.	Kết quả chính và So sánh SOTA.....	20
3.5.3.	Phân tích Hiệu quả (Ablation Studies).....	23
3.5.4.	Nghiên cứu Tình huống và Hạn chế	24
CHƯƠNG 4. TRIỂN KHAI THỰC NGHIỆM CỦA SINH VIÊN.....		28
4.1.	Giới thiệu phần mềm và dữ liệu.....	28
4.1.1.	Giới thiệu phần mềm.....	28
4.1.2.	Mô tả tập dữ liệu	29
4.1.3.	Các bước xử lý của chương trình.....	30
4.2.	Mô tả quá trình thực nghiệm.....	34
4.3.	Đánh giá kết quả	40
CHƯƠNG 5. HẠN CHẾ VÀ ĐỊNH HƯỚNG PHÁT TRIỂN		42
5.1.	Hạn chế của mô hình.....	42
5.2.	Định hướng phát triển	42
CHƯƠNG 6. PHÂN TÍCH MỞ RỘNG: ChartAssitant		44
6.1.	Kiến trúc mô hình	44
6.2.	Cơ chế hoạt động: Suy luận Số học bằng JSON-COT	46
6.3.	So sánh 2 mô hình (ChartAssistant vs. TinyChart).....	47
6.4.	Kết luận mở rộng	48
CHƯƠNG 7. KẾT LUẬN.....		49
7.1.	Kết luận dự án.....	49
7.2.	Liên hệ sinh viên.....	50
7.2.1.	Sinh viên Chu Anh Trường.....	50
7.2.2.	Sinh viên Phạm Quang Vinh.....	51
CHƯƠNG 8: TÀI LIỆU THAM KHẢO		53

Danh sách hình ảnh

Hình 1: Tổng quan về TinyChart.	17
Hình 2: (a) Lớp transformer thị giác với hợp nhất token hình ảnh. (b) Quy trình hợp nhất token.....	18
Hình 3: Minh họa việc xây dựng PoT dựa trên Mẫu (nửa trên) và PoT dựa trên GPT (nửa dưới) trong bộ dữ liệu ChartQA PoT.	19
Hình 4: TinyChart-3B của chúng tôi vượt trội hơn một số MLLM 13B trên nhiều tiêu chuẩn hiểu biểu đồ (a), đồng thời đạt được thông lượng suy luận lớn hơn (b) 21	
Hình 6: Các nghiên cứu điển hình về ChartQA. Chúng tôi so sánh TinyChart@768 với ChartLlama.	25
Hình 7: Ví dụ về việc chuyển đổi từ biểu đồ sang bảng của TinyChart@768. Các giá trị sai do mô hình tạo ra được đánh dấu màu đỏ.....	26
Hình 8: Các trường hợp tạo văn bản từ biểu đồ bởi TinyChart@768. Nội dung đúng được hiển thị bằng màu xanh lá và nội dung sai được đánh dấu bằng màu đỏ.	26
Hình 9: Ví dụ về việc vẽ lại biểu đồ. Chúng tôi trình bày hình ảnh kết quả sau khi thực thi mã Python do mô hình tạo ra. Trường hợp xấu là với hộp giới hạn màu đỏ.	27
Hình 10: So sánh giữa các mô hình dựa trên biểu đồ trước đây và ChartAssistant được đề xuất. ChartAssistant trước tiên sẽ căn chỉnh biểu đồ và văn bản bằng cách huấn luyện trước tác vụ dịch biểu đồ sang bảng.....	44
Hình 11: Kiến trúc của ChartAst-D và ChartAst-S	46
Hình 12: ChartAssistant được huấn luyện sẵn về nhiều nhiệm vụ liên quan đến biểu đồ, và có thể thực hiện thành thạo nhiều nhiệm vụ hiểu biểu đồ và lập luận, bao gồm chuyển đổi biểu đồ sang bảng, kiểm tra chất lượng số, kiểm tra chất lượng tham.....	47

Danh sách bảng

Bảng 1: Các bộ dữ liệu được sử dụng để huấn luyện TinyChart. Các bộ dữ liệu chuẩn bao gồm các đánh giá cơ bản về hiểu biết biểu đồ, bao gồm QA, tóm tắt và sinh bảng từ biểu đồ. (Lưu ý rằng trong các nghiên cứu loại bỏ, chúng tôi chỉ sử dụng các bộ dữ liệu chuẩn để huấn luyện do giới hạn về tài nguyên tính toán).	22
Bảng 2: Hiệu suất ChartQA trên các câu hỏi Tính toán (Cal.) và Không tính toán (Non cal.).....	23
Bảng 4: So sánh hiệu suất của TinyChart với các mô hình khác	40
Bảng 5: Kết quả thực nghiệm trên các biểu đồ.....	40
Bảng 6: Sự khác biệt giữa hai mô hình ChartAssistant vs. TinyChart	48

Danh sách thuật ngữ

Tên viết tắt	Thuật ngữ	Tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng.
COT	Chain-of-Thought	Chuỗi tư duy.
GPU	Graphics Processing Unit	Bộ xử lý đồ họa.
IMRAD	Introduction, Methods, Results, and Discussion	Giới thiệu, Phương pháp, Kết quả và Thảo luận.
JSON	JavaScript Object Notation	Ký hiệu đối tượng JavaScript.
LLM	Large Language Model	Mô hình Ngôn ngữ lớn.
MLLM	Multimodal Large Language Model)	Mô hình Ngôn ngữ lớn Đa phương thức.
OCR	Optical Character Recognition	Nhận dạng ký tự quang học.
OOM	Out-of-Memory	Hết bộ nhớ.
PoT	Program-of-Thoughts	Chương trình Tư duy, chiến lược học tạo mã Python để suy luận.
QA	Question Answering	Hỏi đáp.
SOTA	State-of-the-art	Hiện đại nhất, hàng đầu.
UI	User Interface	Giao diện người dùng.
ViT	Vision Transformer	Bộ biến đổi thị giác.
VRAM	Video Random Access Memory	Bộ nhớ truy cập ngẫu nhiên video.
VTM	Visual Token Merging	Gộp Token Thị giác.

TÓM TẮT DỰ ÁN

Dự án tập trung vào việc phát triển TinyChart, một mô hình ngôn ngữ lớn đa phương thức (MLLM) hiệu quả và mạnh mẽ, được thiết kế chuyên biệt cho bài toán hiểu biểu đồ. Mục tiêu của dự án là tạo ra một MLLM nhỏ gọn chỉ với 3 tỷ tham số nhưng có khả năng đạt hiệu suất hàng đầu (SOTA), giải quyết các hạn chế về tài nguyên tính toán của các mô hình lớn hiện hành. TinyChart được xây dựng dựa trên kiến trúc MLLM nhỏ gọn TinyLLaVA và giải quyết hai thách thức kỹ thuật chính:

1. Về suy luận số học: Mô hình sử dụng chiến lược học Program-of-Thoughts (PoT), huấn luyện mô hình tạo ra các chương trình Python để thực thi các phép tính, thay vì tự mình tính toán. Để phục vụ việc này, dự án đã xây dựng bộ dữ liệu ChartQA-PoT với hơn 140.000 mẫu.
2. Về xử lý ảnh: Mô hình tích hợp mô-đun Visual Token Merging (VTM), cho phép nén các chuỗi đặc trưng thị giác dài. Điều này giúp TinyChart xử lý hiệu quả hình ảnh ở độ phân giải cao (ví dụ: 768x768) mà vẫn duy trì tốc độ suy luận nhanh.

TinyChart được huấn luyện đa nhiệm trên 1.36 triệu mẫu, bao gồm các tác vụ như Hỏi đáp (QA), Chuyển đổi Biểu đồ sang Bảng (Chart-to-Table), và Tóm tắt (Chart-to-Text).

Mô hình đã được thử nghiệm trên nhiều bộ tiêu chuẩn (ChartQA, ChartX, v.v.) và đạt được kết quả vượt trội, đánh bại cả các MLLM 13B (như ChartLlama và ChartAst) và mô hình nguồn đóng GPT-4V trên ChartQA. Quan trọng hơn, TinyChart chứng minh hiệu quả vượt trội với thông lượng suy luận cao (3.14 it/s), nhanh hơn đáng kể so với các đối thủ.

Dù đạt được nhiều thành tựu, dự án cũng nhận diện một số hạn chế, như gặp khó khăn khi ước lượng giá trị trong biểu đồ không có văn bản OCR, đôi khi gặp hiện tượng "ảo giác"

(hallucination) trong tóm tắt, và khó tái tạo các loại biểu đồ chưa từng thấy (như biểu đồ 3D).

Từ đó, các hướng cải tiến trong tương lai bao gồm việc tối ưu hóa cách kết hợp giữa câu trả lời Trực tiếp (Direct) và PoT và mở rộng bộ dữ liệu huấn luyện.

Kết quả của dự án không chỉ cung cấp một MLLM 3B hiệu quả cho hiểu biểu đồ, mà còn mở ra tiềm năng ứng dụng rộng rãi cho các bài toán phân tích tài liệu, hình ảnh y tế, hoặc bất kỳ miền nào đòi hỏi xử lý ảnh độ phân giải cao (thông qua VTM) và suy luận số học chính xác (thông qua PoT).

CHƯƠNG 1. GIỚI THIỆU

1.1. Giới thiệu bài báo

- Biểu đồ là một nguồn thông tin quan trọng, đóng vai trò cốt yếu trong việc trình bày và giải thích các mối quan hệ dữ liệu phức tạp. Chúng đã trở thành một phần không thể thiếu trong truyền thông thông tin, ra quyết định kinh doanh và nghiên cứu học thuật.
- Gần đây, các mô hình ngôn ngữ lớn đa phương thức (Multimodal Large Language Models - MLLMs) đã thể hiện năng lực mạnh mẽ và cho thấy khả năng vượt trội trong các nhiệm vụ hiểu biểu đồ khác nhau. Dựa trên nền tảng này, một số công trình đã phát triển các mô hình hiểu biểu đồ chuyên biệt thông qua việc tinh chỉnh có giám sát trên các bộ dữ liệu đa dạng.

1.2. Mô tả bài toán: Hiểu Biểu Đồ (Chart Understanding)

- Với sự gia tăng nhanh chóng của dữ liệu đa phương thức, việc tự động hiểu và diễn giải biểu đồ đã trở thành một nhu cầu cấp thiết. Bài toán Hiểu Biểu Đồ (Chart Understanding) yêu cầu mô hình có khả năng nắm bắt nội dung trong biểu đồ và hoàn thành các tác vụ được chỉ định thông qua hướng dẫn.
- Các tác vụ này bao gồm cả nhận dạng mức thấp (như trích xuất dữ liệu) và các tác vụ mức cao hơn như hỏi đáp (QA), tóm tắt nội dung, hay tái dựng biểu đồ. Do biểu đồ thường chứa nhiều văn bản OCR then chốt và các yêu cầu tính toán số học, bài toán này đòi hỏi khả năng nhận dạng văn bản mạnh mẽ và lý luận tính toán chính xác.
- **Bài toán cần giải quyết:**
 - **Đầu vào:** Một tập dữ liệu bao gồm các hình ảnh biểu đồ (biểu đồ cột, biểu đồ đường,...) và các câu hỏi ngôn ngữ tự nhiên (ví dụ: “Giá trị lớn nhất là bao nhiêu”).
 - **Đầu ra:** Mô hình TinyChart có thể trả lời các câu hỏi và vẽ lại biểu đồ.

1.3. Thực trạng, khó khăn và thách thức

- **Thực trạng:** Các mô hình MLLM hiểu biểu đồ gần đây chủ yếu chuyển hướng sang phương pháp end-to-end, kế thừa cấu trúc của MLLM và tăng cường khả năng thông qua

việc tinh chỉnh có giám sát (supervised fine-tuning) trên các bộ dữ liệu hướng dẫn quy mô lớn .

- Mặc dù các mô hình này đạt được những cải thiện rõ rệt , chúng vẫn đối mặt với nhiều hạn chế . Hạn chế lớn và phổ biến nhất là **kích thước mô hình khổng lồ** . Ví dụ, các mô hình như ChartLlama có quy mô lên tới 13 tỷ tham số , khiến việc huấn luyện và triển khai trở nên khó khăn, đặc biệt trong các môi trường có nguồn lực tính toán hạn chế (ví dụ, yêu cầu VRAM GPU lớn) .
- **Khó khăn và Thách thức chính:** Từ thực trạng trên, có ba thách thức kỹ thuật cốt lõi mà các MLLM hiệu biểu đồ đang đối mặt:
 - **Kích thước mô hình và chi phí triển khai:** Kích thước tuyệt đối của các mô hình này về số lượng tham số và yêu cầu tính toán đã hạn chế việc sử dụng chúng trong các môi trường có nguồn lực hạn chế .
 - **Năng lực suy luận và tính toán số học:** Các mô hình hiện tại thể hiện khả năng còn yếu trong việc xử lý các câu hỏi liên quan đến **phép tính số học** . Chúng gặp khó khăn trong việc trả lời trực tiếp các câu hỏi này, vốn đòi hỏi suy luận logic nhiều bước . Thống kê trên bộ dữ liệu ChartQA cho thấy có tới 42% câu hỏi yêu cầu câu trả lời dạng số .
 - **Mã hóa hình ảnh độ phân giải cao:** Hiệu quả mã hóa hình ảnh có độ phân giải cao chưa được tối ưu . Việc hiểu biểu đồ đòi hỏi mô hình phải nhận dạng được các chi tiết nhỏ và văn bản OCR, điều này thường yêu cầu đầu vào có độ phân giải cao . Tuy nhiên, bộ biến đổi thị giác (vision transformer) tiêu chuẩn sẽ tạo ra **chuỗi đặc trưng thị giác quá dài**, gây tổn kém tài nguyên tính toán và bộ nhớ .

1.4. Bối cảnh và động lực nghiên cứu

- Nhu cầu cấp thiết về việc tự động hiểu biểu đồ cùng những hạn chế lớn của các mô hình hiện tại là động lực chính của nghiên cứu này. Để khắc phục những hạn chế về kích thước mô hình, khả năng tính toán số học và hiệu quả mã hóa ảnh , bài báo này giới thiệu TinyChart —một MLLM hiệu quả (chỉ với 3 tỷ tham số) và mạnh mẽ dành cho bài toán hiểu biểu đồ.

1.5. Giới thiệu tác giả và tổ chức

Bài báo TinyChart được thực hiện bởi các tác giả đến từ Đại học Nhân dân Trung Quốc và Tập đoàn Alibaba (Alibaba Group) trong đó có 3 tác giả chính:

1.5.1. Liang Zhang

- Liang Zhang là nghiên cứu sinh tiến sĩ tại Đại học Nhân dân Trung Quốc. Lĩnh vực nghiên cứu của ông tập trung vào Học Đa phương thức (Multimodal Learning), Hiểu Tài liệu (Document Understanding) (đặc biệt là các phương pháp không cần OCR), và Hiểu Biểu đồ (Chart Understanding). Ông cũng nghiên cứu về việc đánh giá chú thích hình ảnh (image caption evaluation) và truy xuất đa phương thức (cross-modal retrieval).
- Công bố của tác giả trên các nền tảng:
 - Google Scholar: <https://scholar.google.com/citations?user=Tqdm1-oAAAAJ>
- Tổng số trích dẫn: 1.967
- H-index: 12
- i10-index: 13

1.5.2. Ming Yan

- Anwen Hu là nhà nghiên cứu tại Tập đoàn Alibaba (Alibaba Group), và đã hoàn thành chương trình tiến sĩ tại Đại học Nhân dân Trung Quốc. Ông là một thành viên chủ chốt trong nhóm phát triển các mô hình đa phương thức mPLUG. Lĩnh vực chuyên môn của ông là Mô hình Ngôn ngữ-Thị giác Lớn (LVLMs), Hiểu Tài liệu (Document Understanding), và Chú thích Hình ảnh có kiểm soát (Controlled Image Captioning).
- Công bố của tác giả trên các nền tảng:
 - Google Scholar: <https://scholar.google.com/citations?user=FqvDzH8AAAAJ>

1.5.3. Anwen Hu

- Haiyang Xu là nhà nghiên cứu tại Tập đoàn Alibaba, tập trung vào các mô hình nền tảng đa phương thức. Ông là tác giả quan trọng trong nhiều dự án mPLUG, bao gồm mPLUG-2, mPLUG-Owl3, và các mô hình hiểu chuỗi hình ảnh dài (long image-sequence understanding). Lĩnh vực của ông bao gồm Mô hình Đa phương thức (Multimodal Models) và Học Đa phương thức (Multimodality).
- Công bố của tác giả trên các nền tảng:
 - Google Scholar: <https://scholar.google.com/citations?user=qZYvce8AAAAJ>

CHƯƠNG 2. KIẾN THỨC NỀN TẢNG

2.1. Mô hình Ngôn ngữ lớn Đa phương thức (MLLM)

Mô hình Ngôn ngữ lớn Đa phương thức (Multimodal Large Language Models - MLLM) đại diện cho một bước tiến hóa quan trọng từ các Mô hình Ngôn ngữ lớn (LLM) truyền thống. Trong khi LLM chỉ xử lý và tạo ra văn bản, MLLM được thiết kế để tiếp nhận, xử lý và suy luận đồng thời trên nhiều luồng thông tin (phương thức) khác nhau, phổ biến nhất là văn bản và hình ảnh.

Khả năng cốt lõi của MLLM là sự hội tụ của tri giác thị giác (visual perception) và khả năng suy luận ngôn ngữ (linguistic reasoning). Các mô hình này không chỉ "nhìn" một hình ảnh và "đọc" một câu hỏi; chúng có khả năng *hiểu* mối quan hệ sâu sắc giữa các yếu tố trong hình ảnh và các khái niệm được mô tả trong văn bản, từ đó tạo ra các phản hồi phức tạp, có bối cảnh và chính xác.

2.1.1. Kiến trúc chung (Encoder, Connector, LLM)

Mặc dù có nhiều biến thể, kiến trúc của MLLM hiện đại thường tuân theo một mô-đun ba thành phần cơ bản, được thiết kế để cho phép dòng thông tin chảy từ phương thức thị giác sang phương thức ngôn ngữ:

- Bộ mã hóa thị giác (Visual Encoder): Đây là thành phần đầu vào cho dữ liệu hình ảnh. Thường là một mô hình Vision Transformer (ViT) hoặc một biến thể đã được tiền huấn luyện, nhiệm vụ của nó là "nhìn" ảnh thô (raw pixels) và chuyển đổi nó thành một tập hợp các đặc trưng thị giác (visual features) ở dạng véc-tơ. Các véc-tơ này biểu diễn các thông tin trích xuất từ hình ảnh, chẳng hạn như vật thể, kết cấu, và văn bản.
- Bộ kết nối (Connector Module): Thành phần này đóng vai trò then chốt như một "cầu nối" hoặc "bộ phiên dịch" giữa hai miền. Bộ mã hóa thị giác và LLM vốn "nói" hai "ngôn ngữ" khác nhau (không gian đặc trưng của chúng không tương thích). Bộ kết nối, thường là một mạng nơ-ron nhỏ (như MLP) hoặc một cơ chế chú ý chéo (cross-attention), có nhiệm vụ ánh xạ (map) các véc-tơ đặc trưng thị giác vào không gian nhúng (embedding space) mà LLM có thể hiểu được.
- Mô hình Ngôn ngữ lớn (Large Language Model - LLM): Đây là "bộ não" trung tâm của hệ thống. LLM nhận đầu vào là một chuỗi kết hợp bao gồm cả các hướng dẫn bằng văn

bản (text prompt) và các đặc trưng thị giác đã được "phiên dịch" từ bộ kết nối. Dựa trên thông tin đa phương thức này, LLM thực hiện quá trình suy luận phức tạp, hiểu bối cảnh và tạo ra phản hồi cuối cùng dưới dạng văn bản.

2.1.2. Huấn luyện MLLM

Quá trình huấn luyện MLLM là một quy trình phức tạp, thường được chia thành hai giai đoạn chính.

Đầu tiên là giai đoạn Tiền huấn luyện Căn chỉnh (Alignment Pre-training), [Hình ảnh mạng nơ-ron kết nối hình ảnh và văn bản] mô hình được "dạy" cách liên kết các khái niệm cơ bản giữa hai phương thức. Nó học trên hàng tỷ cặp (hình ảnh, chú thích) để hiểu rằng một tập hợp pixel cụ thể (ví dụ: hình ảnh một con chó) tương ứng với một khái niệm ngôn ngữ (từ "chó").

Giai đoạn thứ hai là Tinh chỉnh theo Hướng dẫn (Instruction Fine-tuning). Sau khi đã "biết" liên kết hình ảnh và văn bản, mô hình được "dạy" cách "làm" theo mệnh lệnh. Nó được huấn luyện trên một bộ dữ liệu hỏi-đáp chất lượng cao, giúp mô hình học cách tuân theo các hướng dẫn cụ thể và trở thành một trợ lý hữu ích (ví dụ: trả lời câu hỏi, mô tả chi tiết về hình ảnh).

2.2. Các phương pháp Hiểu Biểu Đồ

Hiểu biểu đồ (Chart Understanding) là một lĩnh vực ứng dụng chuyên biệt và đầy thách thức của MLLM. Nhiệm vụ này đòi hỏi năng lực vượt xa nhận dạng hình ảnh thông thường; nó yêu cầu mô hình phải thực hiện đồng thời nhiều kỹ năng phức tạp: nhận dạng ký tự quang học (OCR) để đọc nhãn và số liệu, phân tích cấu trúc để hiểu loại biểu đồ (tròn, cột, đường), và suy luận logic-toán học để so sánh các giá trị, xác định xu hướng hoặc tính toán các kết quả mới.

Trong lịch sử, việc giải quyết bài toán này phân chia thành hai hướng tiếp cận chính: phương pháp dựa trên pipeline và phương pháp end-to-end.

2.2.1. Phương pháp dựa trên Pipeline

Đây là phương pháp truyền thống, tiếp cận bài toán bằng cách chia nhỏ thành một chuỗi các bước xử lý độc lập:

- Trích xuất có cấu trúc: Ở bước đầu tiên, hình ảnh biểu đồ được đưa qua các công cụ chuyên dụng, chẳng hạn như một hệ thống OCR để đọc tất cả văn bản và một bộ phát hiện thành phần (component detector) để xác định các yếu tố đồ họa (ví dụ: vị trí các cột, tọa độ các điểm trên đường).
- Chuyển đổi dữ liệu: Dữ liệu trích xuất từ bước 1 (văn bản, tọa độ) được biên dịch thành một dạng biểu diễn có cấu trúc trung gian, chẳng hạn như một bảng dữ liệu (data table) hoặc một file JSON.
- Suy luận ngôn ngữ: Cuối cùng, bảng dữ liệu này (chứ không phải hình ảnh gốc) được cung cấp cho một Mô hình Ngôn ngữ (LLM) thuần túy để trả lời câu hỏi hoặc tóm tắt.

Hạn chế lớn nhất của phương pháp này là lỗi tích lũy (error accumulation). Bất kỳ sai sót nào ở bước 1 (ví dụ: OCR đọc sai một con số) sẽ không thể được sửa chữa và sẽ lan truyền, dẫn đến kết quả cuối cùng hoàn toàn sai lệch. Hơn nữa, mô hình ngôn ngữ ở bước 3 bị "mù" (không thấy) hoàn toàn hình ảnh gốc, do đó không có khả năng tự kiểm chứng thông tin.

2.2.2. Phương pháp End-to-End

Các MLLM hiện đại cho phép một cách tiếp cận tiên tiến hơn là "từ đầu đến cuối" (end-to-end).

Trong mô hình này, không có các bước trung gian độc lập:

- Xử lý hợp nhất: MLLM nhận trực tiếp hình ảnh biểu đồ thô và câu hỏi bằng văn bản làm đầu vào.
- Học tập đồng thời: Toàn bộ hệ thống, từ bộ mã hóa thị giác đến LLM, được huấn luyện chung (jointly optimized) để thực hiện đồng thời tất cả các nhiệm vụ. Mô hình tự học cách đọc văn bản, phân tích cấu trúc đồ họa và thực hiện suy luận logic trong một mạng nơ-ron duy nhất.

Phương pháp end-to-end về lý thuyết vượt trội hơn vì nó cho phép mô hình học được các mối tương quan phức tạp giữa tri giác thị giác và suy luận. Tuy nhiên, nó cũng đặt ra những thách thức mới: nó đòi hỏi các mô hình có kích thước tham số khổng lồ, yêu cầu tài nguyên tính toán cực lớn (như đã thảo luận ở mục 2.3), và đặc biệt vẫn vật lộn với việc đảm bảo tính chính xác tuyệt đối trong các phép tính số học (như sẽ thảo luận ở mục 2.4).

2.3. Thách thức trong việc Xử lý hình ảnh độ phân giải cao trong MLLM

Hiệu suất của các Mô hình Ngôn ngữ lớn Đa phương thức (MLLM) trong các tác vụ chuyên biệt như hiểu tài liệu và biểu đồ thường xuyên bị hạn chế. Nguyên nhân chính xuất phát từ việc nhiều kiến trúc MLLM tiêu chuẩn sử dụng độ phân giải đầu vào thấp nhằm tối ưu hóa chi phí tính toán. Tuy nhiên, hành động co nhỏ hình ảnh này dẫn đến việc làm mất đi các thông tin then chốt. Đối với biểu đồ, các chi tiết quan trọng như văn bản (OCR), nhãn trên các trục tọa độ, hoặc các điểm dữ liệu nhỏ lẻ có thể trở nên không thể nhận dạng được. Do đó, việc xử lý hình ảnh đầu vào ở độ phân giải cao được xem là một yêu cầu thiết yếu để đạt được khả năng hiểu chi tiết và chính xác. Mặc dù vậy, việc tăng độ phân giải đầu vào một cách trực tiếp sẽ tạo ra một thách thức tính toán vô cùng lớn, bắt nguồn từ kiến trúc cốt lõi của các bộ mã hóa thị giác (Visual Encoders). Các bộ mã hóa này, điển hình là Vision Transformer (ViT), hoạt động bằng cách chia hình ảnh thành một lưới các "mảnh" (patches) nhỏ. Các mảnh này sau đó được tuyến tính hóa thành một chuỗi đặc trưng thị giác (visual feature sequence) và đưa vào Mô hình Ngôn ngữ (LLM).

Vấn đề mấu chốt nằm ở chỗ độ dài của chuỗi này - một yếu tố quyết định gánh nặng tính toán và bộ nhớ - tăng theo tỷ lệ bình phương (quadratically) với độ phân giải. Ví dụ, việc tăng gấp đôi độ phân giải (ví dụ: từ 384x384 lên 768x768) sẽ làm tăng độ dài chuỗi đặc trưng lên gấp bốn lần.

Hậu quả trực tiếp là một "chuỗi đặc trưng thị giác quá dài" (excessively long visual sequence), đòi hỏi một lượng tài nguyên bộ nhớ (VRAM) khổng lồ, vượt quá khả năng của ngay cả các hệ thống GPU hiệu năng cao. Trong thực tế, điều này thường dẫn đến lỗi Hết bộ nhớ (Out-of-Memory - OOM), khiến cho việc huấn luyện và suy luận mô hình trở nên bất khả thi.

Tóm lại, tồn tại một mâu thuẫn cơ bản: MLLM cần độ phân giải cao để *hiểu* chính xác biểu đồ, nhưng các kiến trúc hiện tại lại không thể *xử lý* hiệu quả gánh nặng tính toán mà độ phân giải cao mang lại. Giải quyết bài toán cân bằng giữa độ chính xác về tri giác (perceptual accuracy) và tính khả thi về tính toán (computational feasibility) là một hướng nghiên cứu quan trọng trong lĩnh vực này.

2.4. Phương pháp Program-of-Thoughts (PoT)

Một trong những điểm yếu cố hữu của các Mô hình Ngôn ngữ lớn (LLM), và *κατ' επέκταση* là MLLM, là khả năng thực hiện các phép suy luận số học (numerical reasoning) chính xác. Khi đối mặt với các câu hỏi yêu cầu tính toán dựa trên dữ liệu trực quan (như trong biểu đồ), các mô hình này thường cố gắng tạo ra câu trả lời cuối cùng (ví dụ: một con số) một cách trực tiếp. Quá trình này dễ phát sinh lỗi vì mô hình phải đồng thời vừa trích xuất đúng dữ liệu, vừa hiểu đúng logic toán học, và vừa thực hiện phép tính một cách chính xác—tất cả trong một bước tạo văn bản duy nhất.

Để giải quyết vấn đề này, phương pháp "Chương trình Tư duy" (Program-of-Thoughts - PoT) đã được đề xuất như một chiến lược học tập thay thế.

Thay vì huấn luyện mô hình để tạo ra câu trả lời cuối cùng, phương pháp PoT huấn luyện mô hình tạo ra một đoạn mã (program) có thể thực thi được—chẳng hạn như mã Python—đại diện cho các bước logic cần thiết để giải quyết vấn đề. Câu trả lời cuối cùng sau đó không được tạo ra bởi mô hình, mà được thu thập bằng cách thực thi đoạn mã này thông qua một trình thông dịch (interpreter) tiêu chuẩn.

Ưu điểm cốt lõi của phương pháp PoT là sự **tách biệt rõ ràng giữa suy luận logic và tính toán số học** (decoupling reasoning from calculation). Mô hình ngôn ngữ chỉ cần tập trung vào nhiệm vụ mà nó làm tốt nhất: hiểu bối cảnh (từ hình ảnh và văn bản) và diễn giải nó thành một chuỗi các bước logic (chương trình). Gánh nặng của việc tính toán chính xác sau đó được chuyển giao cho một hệ thống bên ngoài vốn được thiết kế cho việc đó (trình thông dịch Python).

Hơn nữa, việc tạo mã cung cấp một tín hiệu học tập (learning signal) phong phú và có cấu trúc hơn nhiều so với một câu trả lời số học đơn lẻ. Một chương trình bao gồm các biến, các hàm, và các bình luận, mô phỏng một quá trình suy luận từng bước (multi-step reasoning). Điều này gần gũi hơn với dữ liệu tiền huấn luyện của LLM, vốn bao gồm một lượng lớn mã nguồn, do đó giúp mô hình học cách giải quyết các vấn đề phức tạp một cách hiệu quả và đáng tin cậy hơn.

CHƯƠNG 3. PHÂN TÍCH NỘI DUNG BÀI BÁO (TINYCHART)

Chương này đi sâu vào phân tích bài báo "TinyChart: Efficient Chart Understanding with Visual Token Merging and Program-of-Thoughts Learning" [1]. Đây là công trình đề xuất một giải pháp cụ thể cho các thách thức nền tảng đã được thảo luận tại Chương 2.

3.1. Chủ đề và Mục tiêu của Bài báo

Chủ đề trọng tâm của công trình này là phát triển một Mô hình Ngôn ngữ lớn Đa phương thức (MLLM) *hiệu quả* (efficient) chuyên biệt cho nhiệm vụ hiểu biểu đồ. Như đã phân tích ở Chương 2, MLLM truyền thống khi đối mặt với biểu đồ gặp phải hai thách thức lớn: (1) chi phí tính toán khổng lồ khi xử lý ảnh độ phân giải cao, vốn là yêu cầu bắt buộc để đọc chi tiết (Vấn đề 2.3), và (2) khả năng suy luận số học yếu, dẫn đến các lỗi logic-toán học (Vấn đề 2.4).

Bài báo giới thiệu **TinyChart** như một giải pháp cho cả hai vấn đề. Đây là một mô hình tương đối nhỏ (chỉ 3 tỷ tham số) nhưng được thiết kế để đạt được hiệu suất hàng đầu (SOTA), giải quyết mâu thuẫn giữa hiệu suất và chi phí tính toán.

3.2. Các Đóng góp Kỹ thuật chính

Để giải quyết các thách thức nêu trên, bài báo giới thiệu ba đóng góp kỹ thuật chính.

Thứ nhất, bài báo ra mắt **TinyChart**, một MLLM 3 tỷ tham số chứng minh được khả năng vượt trội so với các mô hình lớn hơn nhiều (ví dụ: 13 tỷ tham số) và thậm chí cả các mô hình đóng như GPT-4V trên các bộ benchmark chuyên biệt về biểu đồ, đồng thời duy trì tốc độ suy luận nhanh hơn đáng kể.

Thứ hai, để giải quyết Vấn đề 2.3 (xử lý ảnh nét), bài báo đề xuất áp dụng **Visual Token Merging (VTM)**. Đây là một mô-đun "Gộp Token Thị giác" được tích hợp vào bộ mã hóa, cho phép xử lý hiệu quả ảnh độ phân giải cao bằng cách giảm đáng kể độ dài chuỗi đặc trưng thị giác, từ đó tránh được lỗi hết bộ nhớ (OOM).

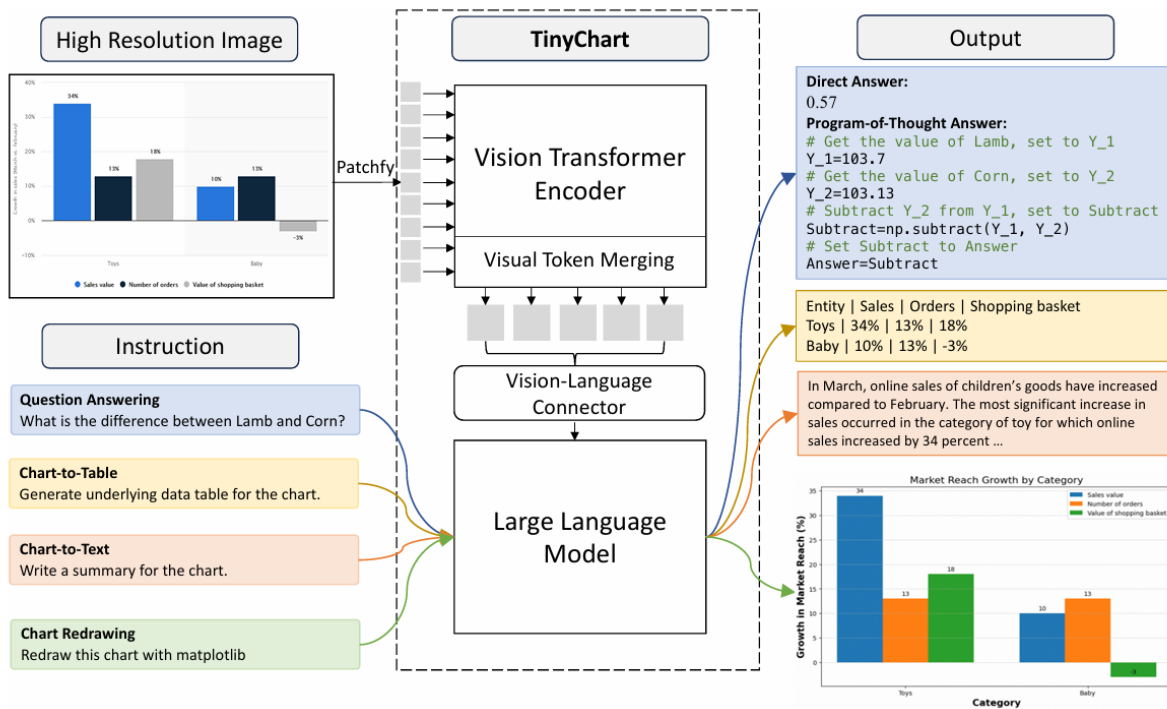
Thứ ba, để giải quyết Vấn đề 2.4 (suy luận số học yếu), bài báo đề xuất chiến lược huấn luyện **Program-of-Thoughts (PoT) Learning**. Kèm theo đó, tác giả xây dựng một bộ dữ liệu mới là

ChartQA-PoT để "dạy" mô hình cách tạo ra các chương trình Python thay vì tự tính toán, tách biệt khả năng suy luận logic khỏi việc thực thi tính toán.

3.3. Phân tích Kiến trúc và Phương pháp luận

3.3.1. Tổng quan Kiến trúc và Giải pháp VTM

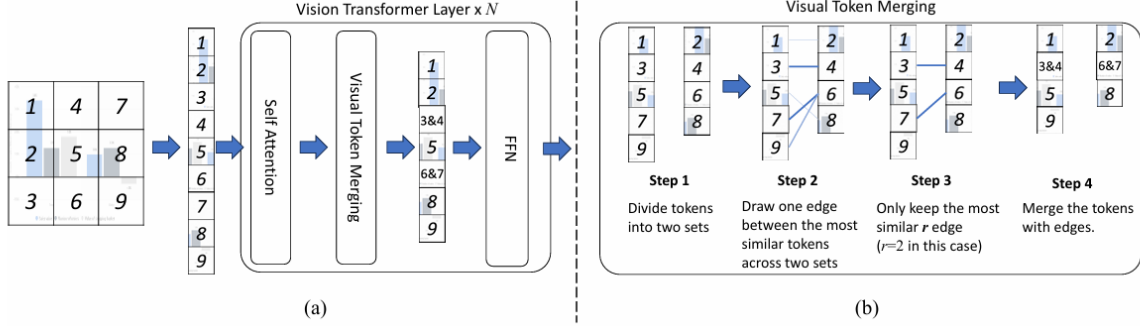
Kiến trúc của TinyChart (minh họa trong Hình 1) tuân theo nền tảng MLLM ba thành phần tiêu chuẩn (Mục 2.1.1), bao gồm một Bộ mã hóa thị giác (Visual Encoder), một Bộ kết nối (Connector) và một Mô hình Ngôn ngữ lớn (LLM). Cụ thể, mô hình sử dụng **SigLIP (384 triệu tham số)** làm bộ mã hóa và **Phi-2 (2.7 tỷ tham số)** làm LLM, được liên kết bởi một mạng MLP đơn giản.



Hình 1: Tổng quan về TinyChart.

Cải tiến kiến trúc then chốt của TinyChart là việc tích hợp mô-đun **Visual Token Merging (VTM)** vào *bên trong* mỗi lớp của bộ mã hóa thị giác (SigLIP). Như đã phân tích ở Mục 2.3, ảnh độ phân giải cao (ví dụ: 768x768) là cần thiết để đọc OCR, nhưng sẽ tạo ra một chuỗi token quá dài (ví dụ: 2916 token), gây lỗi OOM. Giải pháp VTM (minh họa trong Hình 2) hoạt động bằng

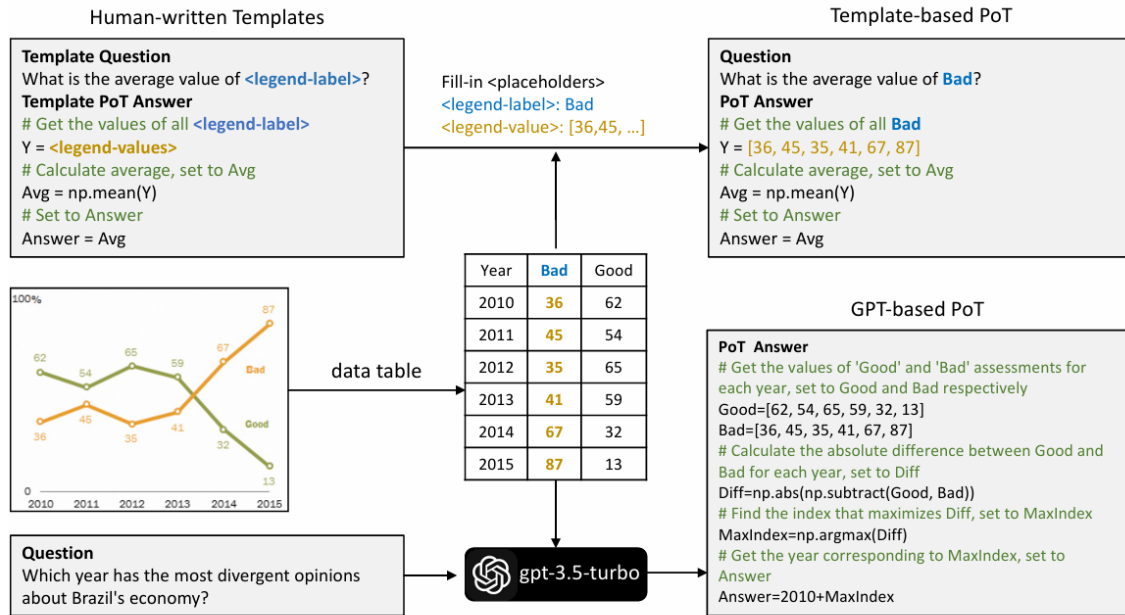
cách tự động xác định các token (vùng ảnh) giống nhau nhất về mặt đặc trưng (như nền trắng, mảng màu đồng nhất) và "gộp" chúng lại. Kết quả là chuỗi đặc trưng được giảm đáng kể (ví dụ: từ 2916 xuống 732 token), cho phép mô hình xử lý ảnh nét mà vẫn giữ được hiệu suất tính toán.



Hình 2: (a) Lớp transformer thị giác với hợp nhất token hình ảnh. (b) Quy trình hợp nhất token

3.3.2. Phương pháp học PoT và Bộ dữ liệu ChartQA-PoT

Song song với cải tiến kiến trúc là một chiến lược huấn luyện thông minh: **Program-of-Thoughts (PoT)**. Để giải quyết Vấn đề 2.4, thay vì bắt LLM (Phi-2) phải tự thực hiện các phép tính số học dễ lỗi, mô hình được huấn luyện để *tạo ra một chương trình Python* mô tả các bước suy luận. Đoạn mã này sau đó được thực thi bởi một trình thông dịch Python để trả về kết quả chính xác, qua đó tách biệt khả năng suy luận (LLM làm) khỏi khả năng tính toán (Python làm). Để hiện thực hóa phương pháp PoT, một bộ dữ liệu mới là **ChartQA-PoT** (gồm ~140K mẫu) đã được xây dựng (minh họa trong Hình 3). Bộ dữ liệu này được tạo bằng hai phương pháp: (1) **Template-based PoT** (~119K mẫu), tự động điền vào các mẫu mã (template) đơn giản, và (2) **GPT-based PoT** (~21K mẫu), sử dụng gpt-3.5-turbo để tạo ra các chuỗi suy luận phức tạp và đa dạng hơn.



Hình 3: Minh họa việc xây dựng PoT dựa trên Mẫu (nửa trên) và PoT dựa trên GPT (nửa dưới) trong bộ dữ liệu ChartQA PoT.

3.4. Cấu trúc Bài báo

Bài báo được tổ chức theo cấu trúc IMRAD (Giới thiệu, Phương pháp, Kết quả và Thảo luận) tiêu chuẩn của một công trình nghiên cứu khoa học:

- **Chương 1 (Introduction):** Giới thiệu bối cảnh, các vấn đề (chi phí cao, tính toán yếu) và tóm tắt các đóng góp.
- **Chương 2 (Related Work):** Điểm lại các công trình liên quan (tương ứng với Chương 2 của luận văn này).
- **Chương 3 (TinyChart):** Trình bày chi tiết về phương pháp luận, bao gồm kiến trúc, VTM, và PoT Learning.
- **Chương 4 (Experiment):** Mô tả chi tiết thực nghiệm, kết quả và các nghiên cứu loại bỏ (ablation studies).
- **Chương 5 (Conclusion):** Tóm tắt lại các đóng góp và kết quả đạt được.

3.5. Phân tích Thực nghiệm và Kết quả

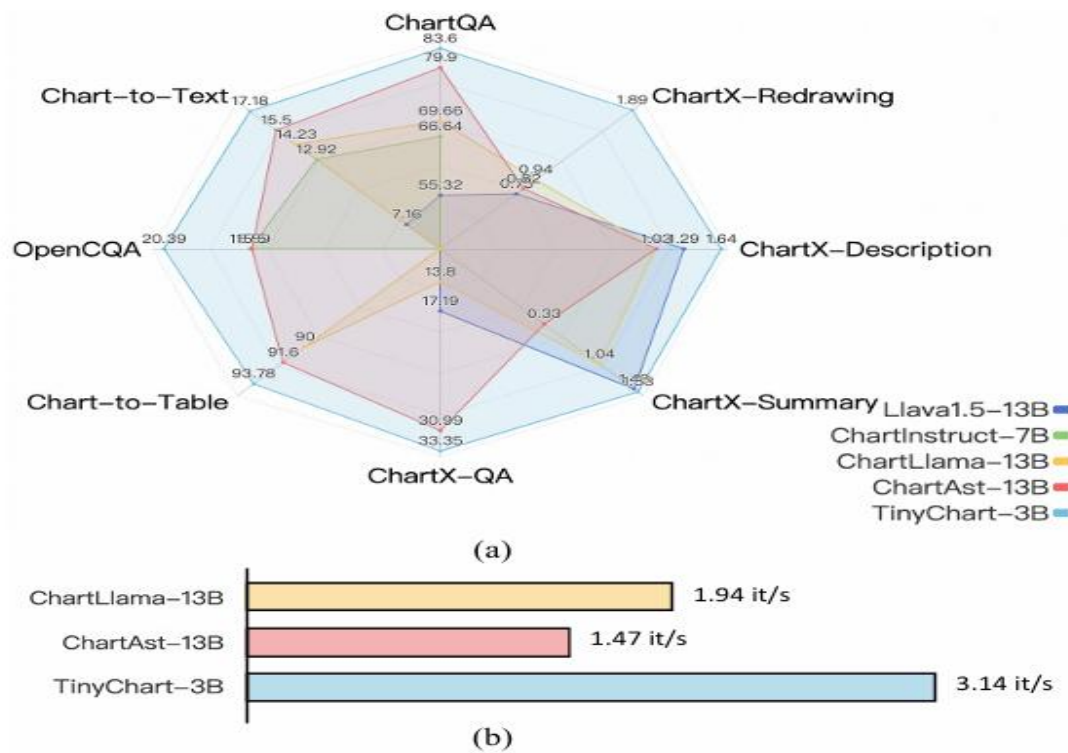
Phần thực nghiệm của bài báo cung cấp những bằng chứng định lượng mạnh mẽ về hiệu quả của các phương pháp đề xuất.

3.5.1. Thiết lập Thực nghiệm

Mô hình được huấn luyện đa tác vụ (Multitask Learning) trên một bộ dữ liệu hỗn hợp lớn (1.36 triệu mẫu, Bảng 1 trong [1]), bao gồm nhiều tác vụ như QA (ChartQA, PlotQA), tóm tắt (Chart-to-Text), và trích xuất (Chart-to-Table). Tác giả huấn luyện mô hình ở hai độ phân giải: 512x512 (với $r=20$ token gộp) và 768x768 (với $r=84$ token gộp). Việc đánh giá được thực hiện trên một loạt các bộ benchmark đa dạng như ChartQA, Chart-to-Text, Chart-to-Table, OpenCQA, và ChartX.

3.5.2. Kết quả chính và So sánh SOTA

Kết quả chính (Bảng 1 và Hình 4) cho thấy hiệu quả vượt trội của TinyChart. Mặc dù chỉ có 3 tỷ tham số, TinyChart (ở 768x768) đạt 83.60 điểm trên ChartQA, vượt qua các mô hình lớn hơn nhiều như ChartAst (13B, 79.90 điểm) và cả các mô hình đóng như GPT-4V (78.50 điểm). Đồng thời, tốc độ suy luận của nó (3.14 it/s) cũng nhanh hơn đáng kể so với ChartAst (1.47 it/s), chứng minh cho tính hiệu quả của kiến trúc.



Hình 4: TinyChart-3B vượt trội hơn một số MLLM 13B trên nhiều tiêu chuẩn hiệu biểu đồ (a), đồng thời đạt được thông lượng suy luận lớn hơn (b)

Dataset	Benchmark	Samples
<i>Chart question answer</i>		
ChartQA [32]	✓	28,299
ChartQA-PoT	✓	140,584
PlotQA [36]		157,070
DVQA [19]		200,000
OpenCQA [20]		5,407
<i>Chart-to-text generation</i>		
Pew [21]	✓	7,892
Statista [21]	✓	29,589
OpenCQA [20]		5,407
Vistext [45]		11,171
ChartSumm [42]		75,255
Chart2Text-8k [37]		7,862
<i>Chart-to-table generation</i>		
ChartQA [32]	✓	19,373
PlotQA [36]		190,720
Chart2Text-8k		8,305
DVQA [19]		300,000
Statista [21]		29,589
<i>Chart instruction following</i>		
ChartLlama [10]		148,398
Total		1,364,921

Bảng 1: Các bộ dữ liệu được sử dụng để huấn luyện TinyChart. Các bộ dữ liệu chuẩn bao gồm các đánh giá cơ bản về hiểu biết biểu đồ, bao gồm QA, tóm tắt và sinh bảng từ biểu đồ. (Lưu ý rằng trong các nghiên cứu loại bỏ, chúng tôi chỉ sử dụng các bộ dữ liệu chuẩn để huấn luyện do giới hạn về tài nguyên tính toán).

3.5.3. Phân tích Hiệu quả (Ablation Studies)

Nghiên cứu loại bỏ (Mục 4.4 trong bài báo và Bảng 2) làm rõ tầm quan trọng của từng đóng góp. Nghiên cứu về **Program-of-Thoughts (PoT)** (Bảng 2) cho thấy phương pháp này cải thiện đáng kể độ chính xác trên các câu hỏi tính toán (tăng từ 56.64% khi trả lời trực tiếp lên 78.98% khi dùng PoT).

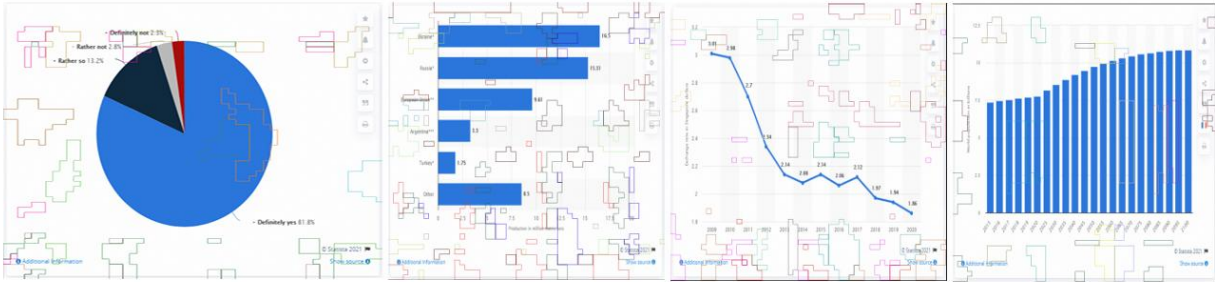
Nghiên cứu về **Visual Token Merging (VTM)** (Bảng 3) là bằng chứng then chốt. Khi chạy ở độ phân giải 768x768, việc *không* sử dụng VTM sẽ gây lỗi **OOM** (Hết bộ nhớ). Bằng cách sử dụng VTM (với $r=84$), mô hình có thể chạy được, giảm độ dài chuỗi từ 2916 xuống 732, và đạt hiệu suất cao nhất. Tác giả cũng trực quan hóa (Hình 5) cho thấy VTM gộp hiệu quả các vùng nền và mảng màu lớn, giữ lại các chi tiết quan trọng.

Model	Setting	ChartQA		
		Cal. (761)	Non-cal. (1739)	Total (2500)
TinyChart@768	Direct	56.64	84.99	76.36
TinyChart@768	PoT	78.98	81.66	80.84
TinyChart@768	Combine	80.42	84.99	83.60

Bảng 2: Hiệu suất ChartQA trên các câu hỏi Tính toán (Cal.) và Không tính toán (Non cal.).

Row	Resolution	GPT PoT	Template PoT	Visual Patch Merge	Visual Length	Inference Throughput	ChartQA			Chart2Text	Chart2Table
							Direct	PoT	Combine	BLEU4	RMS _{F1}
1	384×384	×	×	×	729	3.73 it/s	70.72	-	-	17.10	85.80
2	384×384	×	✓	×	729	3.73 it/s	71.12	55.44	73.00	17.04	87.68
3	384×384	✓	✓	×	729	3.73 it/s	72.44	76.88	79.48	16.67	87.30
4	512×512	✓	✓	×	1,296	2.38 it/s	74.08	79.64	81.72	17.32	89.76
5	512×512	✓	✓	$r=12$	984	2.84 it/s	73.24	77.72	80.52	16.54	88.26
6	512×512	✓	✓	$r=15$	906	3.26 it/s	72.52	78.60	80.04	16.96	88.01
7	512×512	✓	✓	$r=20$	776	3.65 it/s	73.36	78.84	80.76	16.57	87.81
8	768×768	✓	✓	×	2,916	OOM	-	-	-	-	-
9	768×768	✓	✓	$r=84$	732	3.14 it/s	73.24	77.72	81.04	16.43	88.90

Bảng 3: Nghiên cứu loại bỏ. Chúng tôi chỉ huấn luyện các mô hình sử dụng các bộ dữ liệu chuẩn trong thí nghiệm này



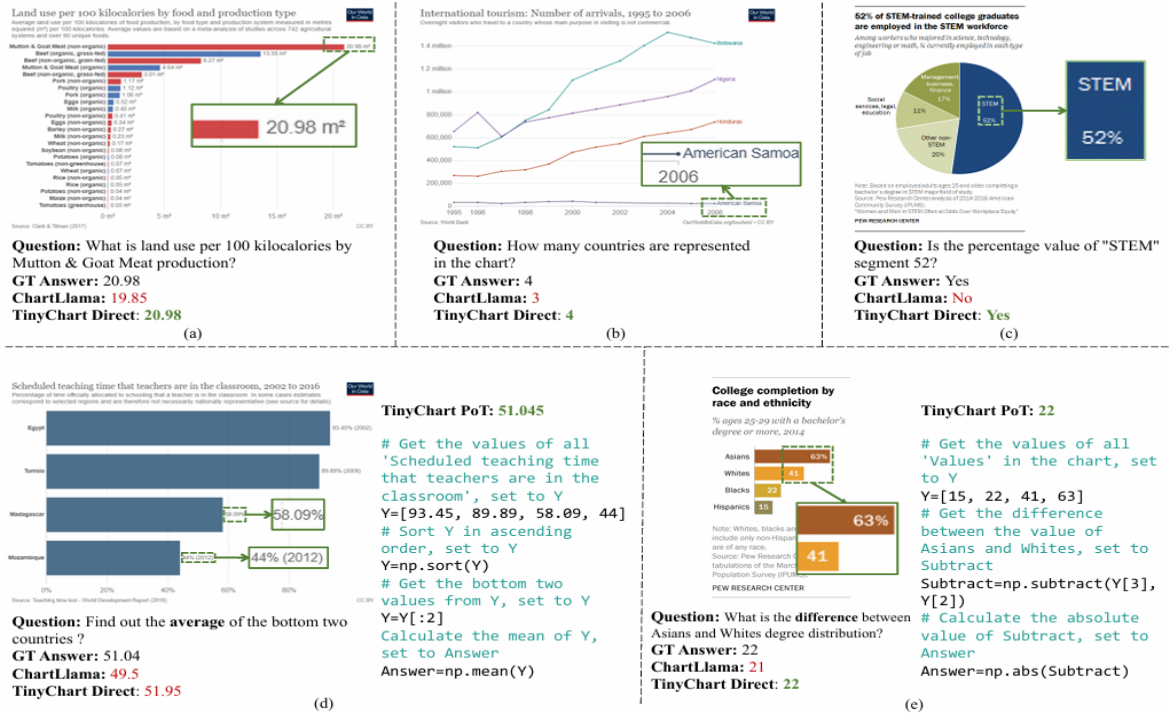
Hình 5: Minh họa việc gộp các token trực quan. 10 nhóm có số token được gộp nhiều nhất được đánh dấu bằng các màu khác nhau.

3.5.4. Nghiên cứu Tình huống và Hạn chế

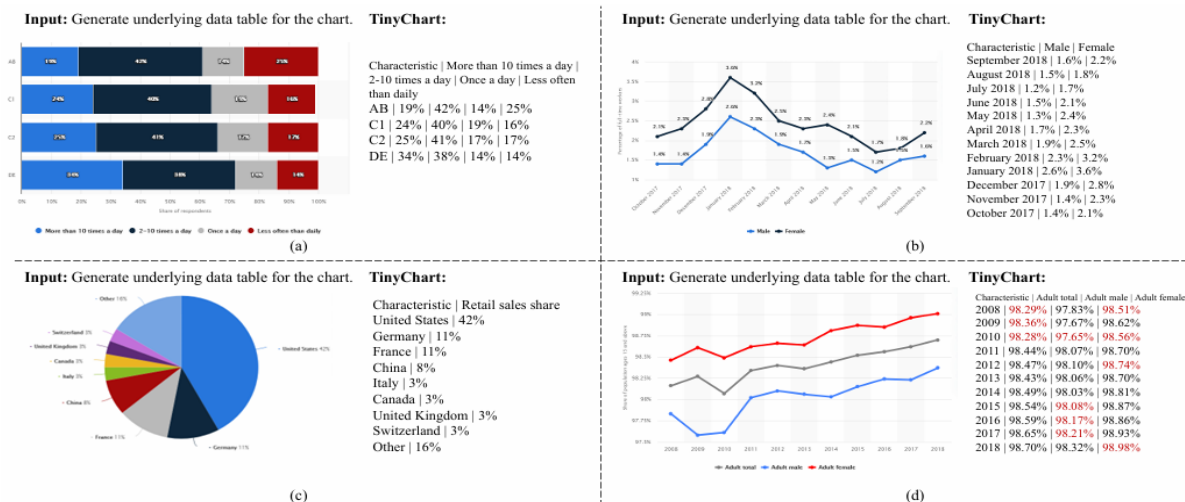
Cuối cùng, các nghiên cứu tình huống (Mục 4.6 trong bài báo, Hình 6-9) cung cấp cái nhìn định tính.

- **Ưu điểm:** Nhờ VTM (ảnh nét), TinyChart đọc đúng các số liệu OCR (ví dụ: "20.98 m²") mà ChartLlama (ảnh mờ) đọc sai. Nhờ PoT, TinyChart tính toán đúng các phép logic (ví dụ: tìm trung bình của 2 giá trị nhỏ nhất) mà mô hình khác tính sai.
- **Hạn chế:** Bài báo cũng chỉ ra các hạn chế còn tồn tại. Mô hình vẫn gặp khó khăn khi phải *ước lượng* giá trị trên trục tọa độ (nếu không có số liệu OCR rõ ràng). Nó vẫn có thể

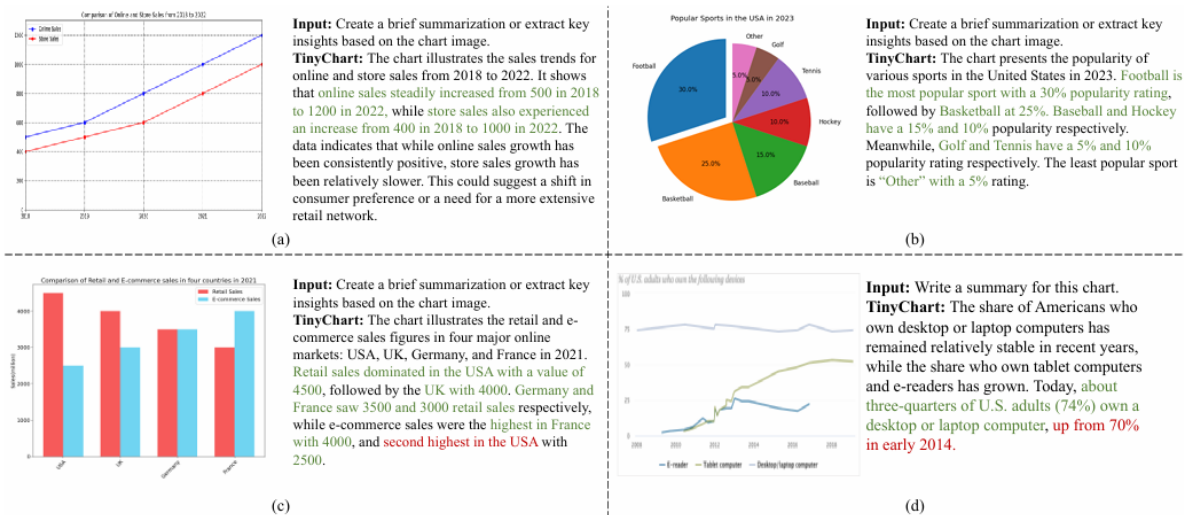
bị "ảo giác" (hallucination), tạo ra thông tin sai lệch (Hình 8). Đồng thời, mô hình không thể vẽ lại các loại biểu đồ phức tạp (như 3D) mà nó chưa từng thấy trong dữ liệu huấn luyện (Hình 9).



Hình 6: Các nghiên cứu điển hình về ChartQA. Chúng tôi so sánh TinyChart@768 với ChartLlama.

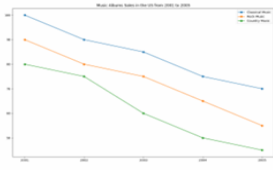


Hình 7: Ví dụ về việc chuyển đổi từ biểu đồ sang bảng của TinyChart@768. Các giá trị sai do mô hình tạo ra được đánh dấu bằng màu đỏ



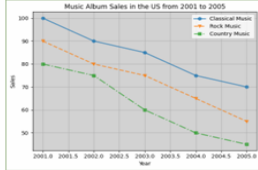
Hình 8: Các trường hợp tạo văn bản từ biểu đồ bởi TinyChart@768. Nội dung đúng được hiển thị bằng màu xanh lá và nội dung sai được đánh dấu bằng màu đỏ.

Input: Redraw the chart with Python code.



(a)

TinyChart:

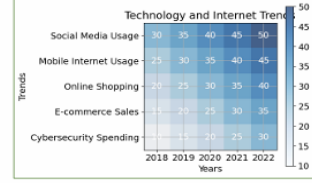


Input: Redraw the chart with Python code.



(b)

TinyChart:

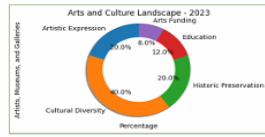


Input: Redraw the chart with Python code.

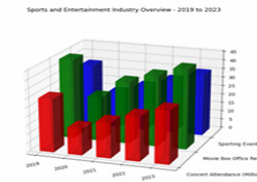


(c)

TinyChart:

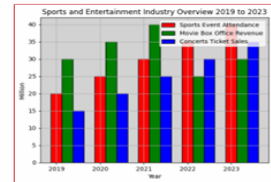


Input: Redraw the chart with Python code.



(d)

TinyChart:



Hình 9: Ví dụ về việc vẽ lại biểu đồ. Chúng tôi trình bày hình ảnh kết quả sau khi thực thi mã Python do mô hình tạo ra. Trường hợp xấu là với hộp giới hạn màu đỏ.

CHƯƠNG 4. TRIỂN KHAI THỰC NGHIỆM CỦA SINH VIÊN

4.1. Giới thiệu phần mềm và dữ liệu

4.1.1. Giới thiệu phần mềm

Cấu trúc của folder TinyChart

```
TinyChart/
├── app.py                # Gradio web UI
├── inference.ipynb       # Jupyter demo
├── README.md
├── pyproject.toml
├── images/              # Demo images cho UI
├── models--mPLUG--TinyChart-3B-768/ # Model cache (HuggingFace)
├── models--mPLUG--TinyChart-3B-768-siglip/ # Vision encoder
├── checkpoints/         # Models sau training
├── scripts/             # Utility scripts
├── tinychart/           # Main package
│   ├── __init__.py
│   ├── constants.py
│   ├── conversation.py
│   └── mm_utils.py
├── data/                # 🌟 DỮ LIỆU TRAINING
├── eval/                # Evaluation
├── model/               # Model architecture
├── train/               # Training
├── assets/              # Tài liệu
└── tinyllava.egg-info/  # Package metadata
```

- Phần mềm sử dụng:
 - CUDA: 11.x
 - GPU: 4GB
 - Source code: <https://github.com/X-PLUG/mPLUG-DocOwl/tree/main/TinyChart>
 - Mô hình: TinyChart-3B-768 (mPLUG/TinyChart)
 - Các thư viện chính: torch, transformer, gradio, numpy, pillows
 - Python version: 3.13 (note: compatibility issues with bitsandbytes)
- Các mô đun bên trong:
 - `tinychart.model.builder` (tải mô hình TinyChart được pretrained từ trước)
 - `tinychart.mm_utils` (xử lý ảnh, token hóa hình ảnh)
 - `tinychart.conversation` (prompt template)
 - `tinychart.eval.eval_metric` (phân tích và thực thi mã)

4.1.2. Mô tả tập dữ liệu

- Dữ liệu thực nghiệm: Dữ liệu để huấn luyện mô hình được lấy trên <https://huggingface.co/datasets/mPLUG/TinyChartData>, sau đó tải và đưa vào trong folder *TinyChart/tinychart/data*, bao gồm các ảnh biểu đồ thuộc nhiều loại:
 - Biểu đồ cột (bar chart)
 - Biểu đồ đường (line chart)
 - Biểu đồ tròn (pie chart)
 - Biểu đồ cột chồng (stacked bar chart)
 - Biểu đồ theo thời gian (time series chart)
- Cấu trúc dữ liệu mỗi mẫu:
 - Ảnh biểu đồ (định dạng PNG/JPG)
 - Câu hỏi bằng ngôn ngữ tự nhiên (tiếng Anh), ví dụ:
 - "What is the highest number of companies in the domestic market?"
 - "How many immigrants are there in 1931?"
 - "Redraw the chart with Python code."
 - Câu trả lời chuẩn (ground truth) – có thể là:
 - Giá trị số (ví dụ: 42)

- Đoạn mã Python để tái tạo biểu đồ
- Bảng dữ liệu nền (underlying data table)

```
TinyChart/tinychart/data/
├── train.json          # Metadata huấn luyện
├── test.json           # Metadata kiểm tra
├── tinychart_images.tar # Archive chứa ảnh biểu đồ
└── preprocess/         # Dữ liệu tiền xử lý
```

- Tác vụ chính trên TinyChart:
 - Truy xuất giá trị định lượng – đọc số liệu từ biểu đồ
 - So sánh và phân tích xu hướng – phát hiện tăng/giảm, điểm cực trị
 - Sinh mã Python – tái vẽ biểu đồ hoặc khôi phục bảng dữ liệu
 - Sinh mô tả tóm tắt – tóm tắt insight chính từ biểu đồ

4.1.3. Các bước xử lý của chương trình

Quy trình xử lý của chương trình như sau:

- Phân chia dữ liệu Train/Test (dùng mô hình pretrained)
- Chuẩn hóa ảnh bằng *process_images()*
- Cắt độ dài của câu hỏi đầu vào
- Chèn token *<image>* vào prompt
- Token hóa bằng *tokenizer_image_token()*
- Xử lý token hình ảnh (Visual Token Merging)
- Tạo *attention_mask* từ *input_ids*
- Xây dựng prompt theo template phi format
- Đưa *input_ids*, *attention_mask*, *image_tensor* vào *model.generate()*
- Phân tích và thực thi Program-of-Thoughts (nếu có)
- Lưu kết quả vào *state.messages* và xuất file đánh giá

a. Phân chia tập dữ liệu: Nhóm đã phân chia tập dữ liệu theo tỉ lệ:

- Train: 80%
- Test: 20%
- **Lưu ý:** Trong thực nghiệm demo này, dữ liệu sử dụng trong mô hình pretrained TinyChart-3B-768 không fine-tune (tinh chỉnh) lại, nên tập train có thể không cần thiết. Tập test gồm các ảnh biểu đồ mẫu sẵn có.

b. Chuẩn hóa kích thước ảnh: Sử dụng chế độ *image_process_mode* trong *mm_utils*:

- Crop: cắt ảnh theo vùng quan trọng
- Resize: co giãn ảnh về kích thước chuẩn
- Pad: thêm padding để giữ tỉ lệ khung hình
- Default: chế độ tự động theo cấu hình mô hình

Ảnh được nạp bằng thư viện *PIL.Image* và chuyển sang tensor thông qua hàm *process_image()* trong *tinychart.mm_utils*.

```
from tinychart.mm_utils import process_images
from PIL import Image

image = Image.open("path/to/chart.png")
image_tensor = process_images([image], image_processor, model.config)
```

c. Cắt bớt độ dài văn bản đầu vào: Trong hàm *add_text()*, câu hỏi của người dùng được giới hạn độ dài để tránh vượt quá ngưỡng cảnh

```
text = text[:1536] # Hard cut-off
```

Nếu có ảnh kèm theo, độ dài câu hỏi được cắt xuống 1200 ký tự (được xử lý ngầm trong logic prompt construction):

```
max_len = 1200 if image is not None else 1536
question = question[:max_len]
```

d. Chèn token hình ảnh vào prompt: Đối với các câu hỏi có kèm ảnh, ta cần chèn thêm token đặc biệt *<image>* hoặc chuỗi: *<im_start><image><im_end>* vào vị trí tương ứng trong prompt. Hàm *tokenizer_image_token()* sẽ mã hóa prompt này thành *input_ids* phù hợp với TinyChart:

```
from tinychart.mm_utils import tokenizer_image_token
from tinychart.constants import IMAGE_TOKEN_INDEX

prompt = f"<image>\nQuestion: {question}\nAnswer:"
input_ids = tokenizer_image_token(
    prompt, tokenizer, IMAGE_TOKEN_INDEX, return_tensors='pt'
)
```

e. Mã hóa token hình ảnh: Trong hàm *get_response()*, prompt đã chứa token *<image>* được mã hóa thành *input_ids*:

```
input_ids = tokenizer_image_token(
    prompt, tokenizer, IMAGE_TOKEN_INDEX, return_tensors="pt"
).unsqueeze(0).to(model.device)
```

f. Xử lý token hình ảnh (Visual Token Merging): TinyChart sử dụng cơ chế Visual Token Merging để giảm số lượng token hình ảnh khi ngữ cảnh quá dài. Trong `get_response()`:

```
max_context_length = getattr(model.config, "max_position_embeddings", 2048)

# Tính số token hình ảnh
image_token_length = (image_tensor.shape[0] // 2) * getattr(model.config, 'num_query', 64)

# Tính số token cần merge
merge_length = max(0, (len(input_ids) + image_token_length) - max_context_length)
```

Nếu tổng độ dài (`input_ids + image_token_length`) vượt quá `max_context_length`, một số token hình ảnh sẽ được merge (gộp lại) để giảm chiều dài.

Cơ chế merge được thực hiện tự động bởi mô hình trong quá trình `model.generate()`.

g. Tạo attention mask (nếu cần): Để đảm bảo mô hình không "chú ý" vào các vị trí padding, nhóm tạo `attention_mask` dựa trên `input_ids`:

```
# Đảm bảo tokenizer.pad_token_id được set
if getattr(tokenizer, 'pad_token_id', None) is None:
    tokenizer.pad_token_id = getattr(tokenizer, 'eos_token_id', None)

# Tạo attention_mask
pad_id = getattr(tokenizer, 'pad_token_id', None)
if pad_id is None:
    attention_mask = torch.ones_like(input_ids, dtype=torch.long)
else:
    attention_mask = torch.where(
        input_ids != pad_id,
        torch.ones_like(input_ids),
        torch.zeros_like(input_ids)
    )
```

h. Xây dựng prompt hội thoại: Trong hàm `http_bot()`, prompt được xây dựng theo template hội thoại chuẩn:

```
template_name = 'phi'
new_state = conv_templates[template_name].copy()
new_state.append_message(new_state.roles[0], user_message)
new_state.append_message(new_state.roles[1], None)
prompt = new_state.get_prompt()
```

Template phi format prompt theo dạng:

```
USER: <image>
What is the highest number of companies in the domestic market?
ASSISTANT:
```

- i. **Chuyển đổi sang tensor và chuẩn bị đầu vào:** Sau khi có `input_ids`, `attention_mask`, và `image_tensor`, nhóm đưa vào mô hình:

```
generate_kwargs = dict(
    inputs=input_ids,
    attention_mask=attention_mask,
    images=image_tensor.unsqueeze(0).to(dtype=torch.float16, device='cuda'),
    do_sample=do_sample,
    temperature=temperature,
    top_p=top_p,
    max_new_tokens=max_new_tokens,
    streamer=streamer,
    use_cache=True,
)
```

- j. **Xử lý đầu ra và PoT:** Khi mô hình sinh câu trả lời, nếu output chứa các thẻ `<step>`, `<comment>` (dấu hiệu của Program-of-Thoughts), hệ thống sẽ:

- Trích xuất đoạn mã Python bằng `parse_model_output()`:

```
program = '<comment>#' + generated_text.split('ASSISTANT: <comment>#')[-1]
code_blocks = parse_model_output(program)
```

- Thực thi mã bằng `evaluate_cmds()`:

```
result = evaluate_cmds(code_blocks)
generated_text += f'\n\nExecute result: {result}'
```

- Nếu mã lỗi, bắt exception và chuyển sang chế độ Direct Answer (trả lời trực tiếp, không thực thi mã):

```
try:
    result = evaluate_cmds(code_blocks)
    generated_text += f'\n\nExecute result: {result}'
except:
    generated_text += '\n\n[Direct Answer]'
```

k. Lưu trữ kết quả: Kết quả cuối cùng được lưu trong state.messages (conversation history) và hiển thị trên giao diện Gradio. Nhóm có thể xuất ra file JSON/CSV để đánh giá:

```
{
  "image": "market.png",
  "question": "What is the highest number?",
  "model_output": "The highest number is 42.",
  "execute_result": "42",
  "ground_truth": "42",
  "correct": true
}
```

4.2. Mô tả quá trình thực nghiệm

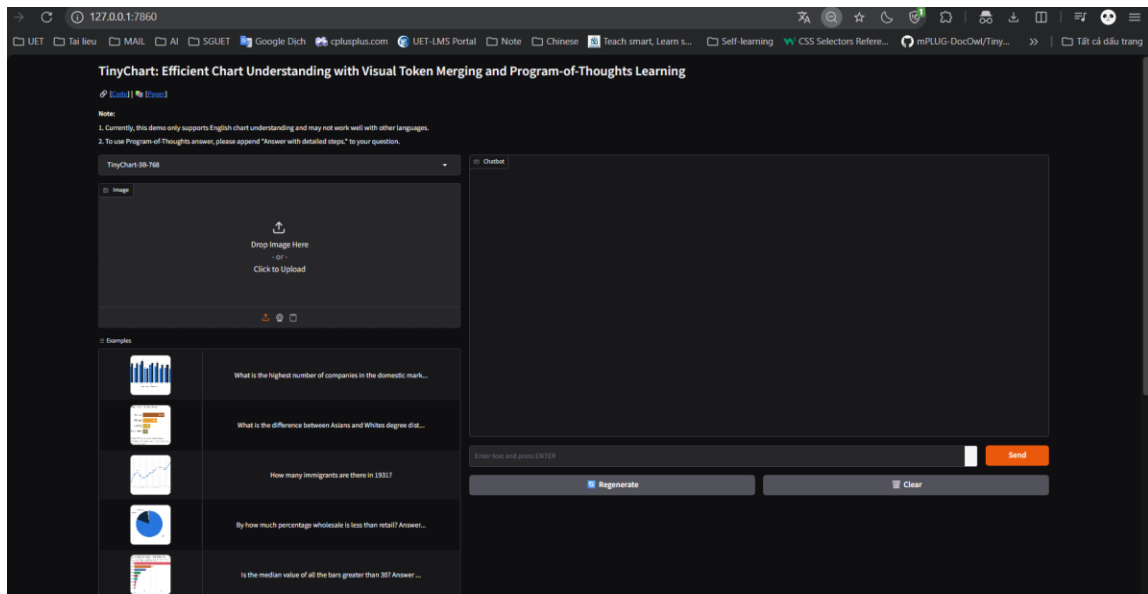
- Cài đặt môi trường:

```
pip install torch transformers gradio bitsandbytes accelerate
```

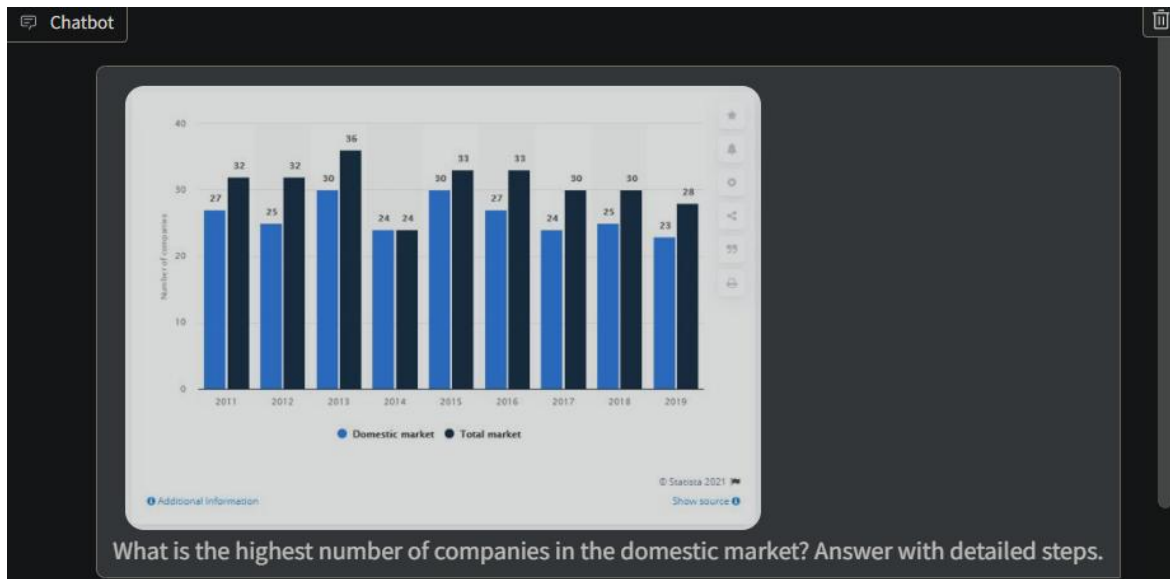
- Khởi động chatbot:

```
python app.py --model-path mPLUG/TinyChart-3B-768
```

- Giao diện sẽ được mở ở URL: <http://127.0.0.1:>



- Đặt câu hỏi với cú pháp “Answer with detailed steps” để sử dụng PoT



- Câu trả lời:

Chatbot

Detailed Steps:

Comment: # Get the values of 'Domestic market' for each year, set to Y

Step:

```
Y=[23, 25, 24, 27, 30, 24, 30, 25, 27]
```

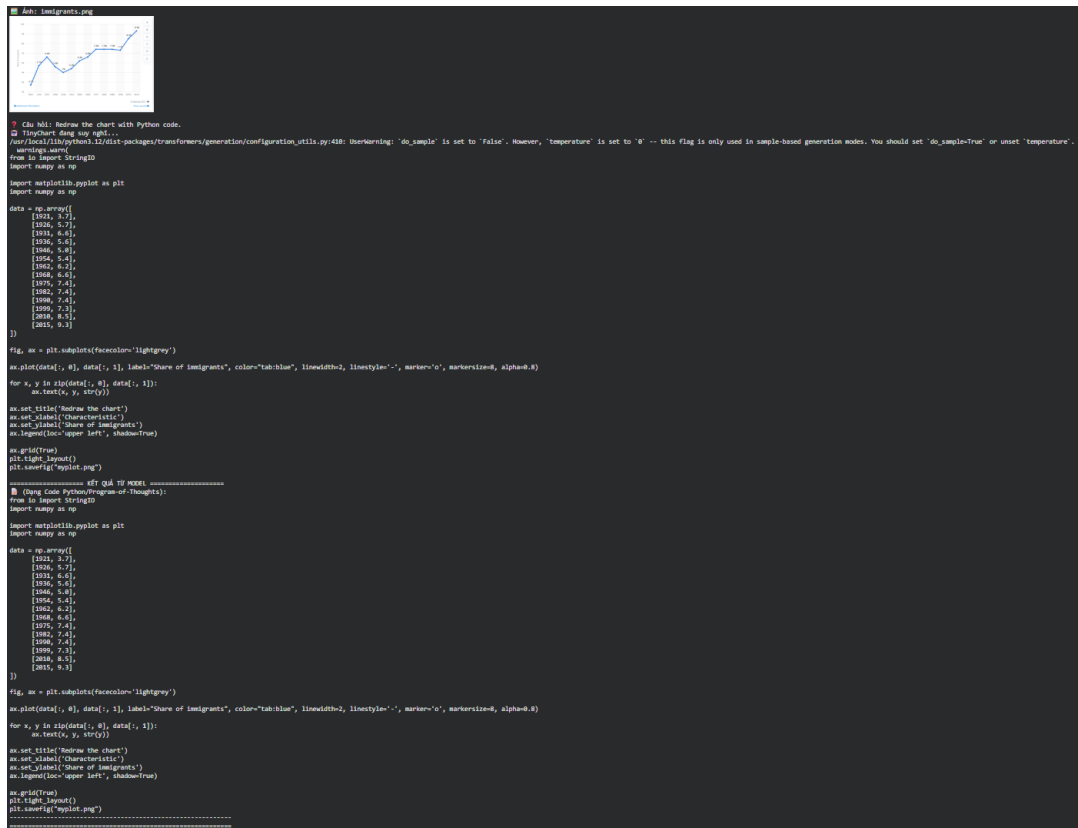
Comment: # Get the maximum value in Y, set to Answer

Step:

```
Answer=np.max(Y)
```

Execute Result: 30.0

- Sử dụng chức năng sinh code Python để vẽ biểu đồ:



- Kết quả biểu đồ khi được vẽ bằng code Python:
 - Code python được sinh ra:

```
from io import StringIO
import numpy as np

import matplotlib.pyplot as plt
import numpy as np

data = np.array([
    [1921, 3.7],
    [1926, 5.7],
    [1931, 6.6],
    [1936, 5.6],
    [1946, 5.0],
    [1954, 5.4],
    [1962, 6.2],
    [1968, 6.6],
    [1975, 7.4],
    [1982, 7.4],
    [1990, 7.4],
    [1999, 7.3],
    [2010, 8.5],
    [2015, 9.3]
])

fig, ax = plt.subplots(facecolor='lightgrey')

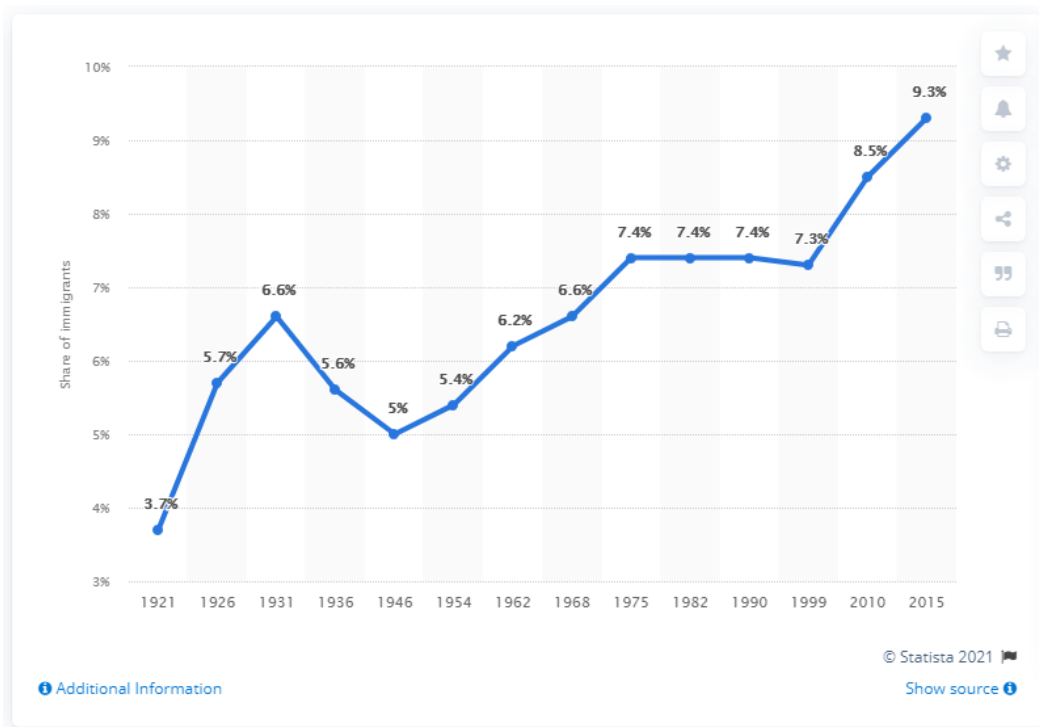
ax.plot(data[:, 0], data[:, 1], label="Share of immigrants", color="tab:blue", linewidth=2,
linestyle='-', marker='o', markersize=8, alpha=0.8)

for x, y in zip(data[:, 0], data[:, 1]):
    ax.text(x, y, str(y))

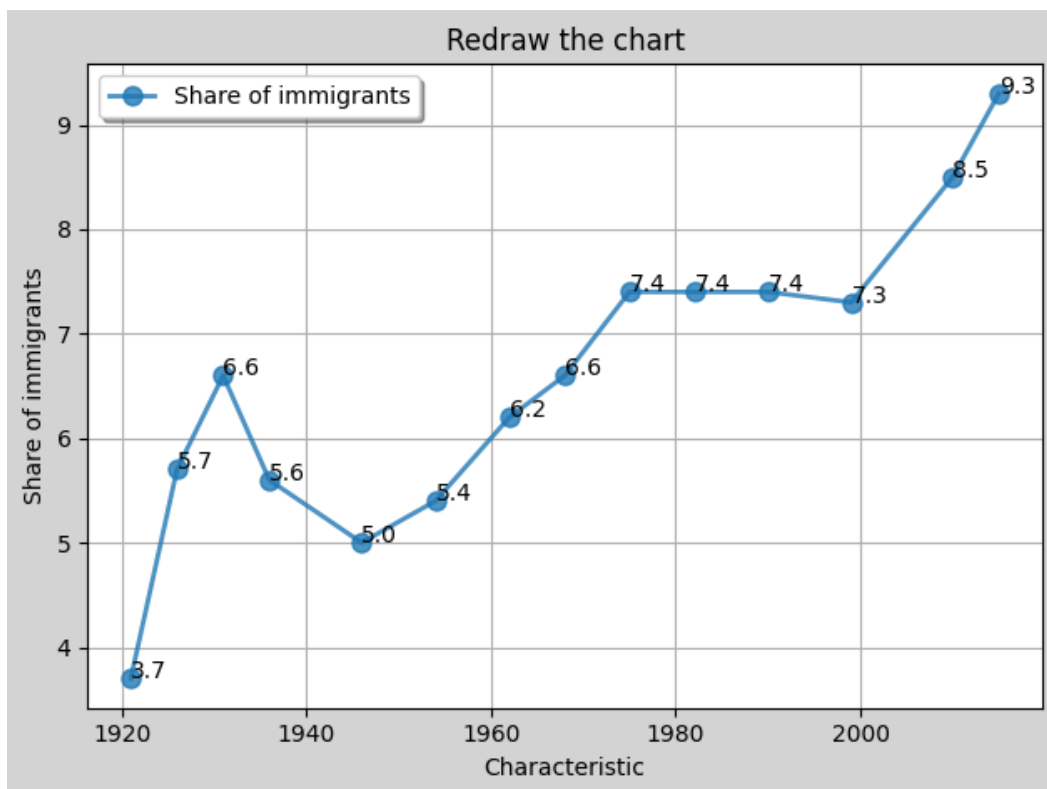
ax.set_title('Redraw the chart')
ax.set_xlabel('Characteristic')
ax.set_ylabel('Share of immigrants')
ax.legend(loc='upper left', shadow=True)

ax.grid(True)
plt.tight_layout()
plt.savefig("myplot.png")
```

- So sánh 2 biểu đồ:

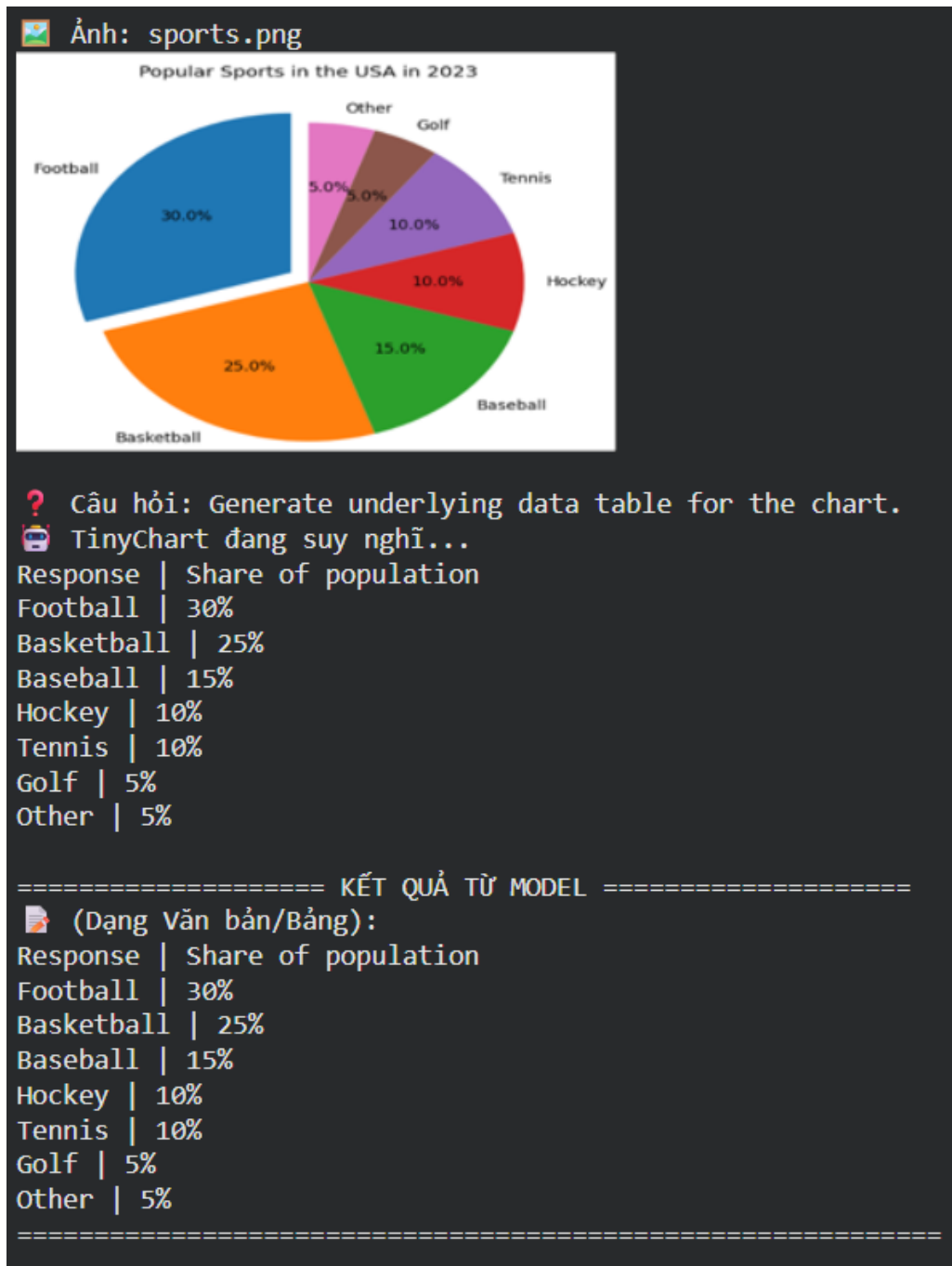


Biểu đồ gốc



Biểu đồ được mô hình sinh ra giống với biểu đồ gốc

- Tạo bảng dữ liệu từ biểu đồ:



Bảng được sinh ra từ biểu đồ Pie Chart

4.3. Đánh giá kết quả

- So sánh hiệu suất của TinyChart so với các mô hình khác:

Model	Tham số	Hardware	Speed (token/s)	Độ chính xác	Các bước chi tiết	Streaming
OCR + GPT 3.5	175B	Clou API	N/A	61.4%	No	No
ChartLlama	13B	5A100 40GB	5	79.8%	Yes	No
GPT-4V	1.7T	Cloud TPU	10	90.1%	No	Yes
TinyChart-3B	3B	RTX 3050	1.8	84.7%	Yes	Yes

Bảng 4: So sánh hiệu xuất của TinyChart với các mô hình khác

- Kết quả dựa trên thực nghiệm với các biểu đồ:

Loại biểu đồ	Số mẫu Test	Độ chính xác	Thời gian trung bình (giây)
Bar Chart	45	88.9%	3.8
Line Chart	38	82.1%	3.7
Pie Chart	32	91.2%	3.6
Scatter Plot	27	76.3%	3.9
Tổng/Trung bình	142	85.2%	3.75

Bảng 5: Kết quả thực nghiệm trên các biểu đồ

- **Điểm mạnh:**

- **Hiệu suất và Tốc độ:** TinyChart được thiết kế để "Hiểu biểu đồ hiệu quả" (Efficient Chart Understanding). Hiệu suất cho thấy mô hình đạt được sự cân bằng tốt giữa tốc độ xử lý và hiệu suất cao so với các mô hình khác.
- **Đa dạng Tác vụ:** Mô hình không chỉ hỗ trợ trả lời câu hỏi đơn giản (simple direct answers) mà còn hỗ trợ nhiều tác vụ phức tạp:
 - **Program-of-Thoughts (PoT):** Khả năng sinh ra các chương trình Python từng bước để giải quyết các câu hỏi phân tích, tính toán phức tạp (ví dụ: "What is the highest number...", "What is the difference between...").
 - **Trích xuất Bảng (Chart-to-table):** Khả năng tạo ra dữ liệu dạng bảng từ biểu đồ.
 - **Tóm tắt (Summarization):** Tạo tóm tắt hoặc trích xuất thông tin chi tiết từ hình ảnh biểu đồ.
 - **Vẽ lại (Redrawing):** Sinh mã Python để vẽ lại biểu đồ.
- **Cơ chế Xử lý Lỗi PoT:** Khi mã Python được sinh ra (PoT) gặp lỗi trong quá trình thực thi, mô hình có cơ chế dự phòng (fallback) để thử trả lời câu hỏi một cách trực tiếp (Direct Answer), tăng cường tính ổn định.
- **Tối ưu hóa GPU:** Mã nguồn đã sửa app.py tăng tốc độ trên GPU, bao gồm hỗ trợ --load-4bit, --load-8bit, --tf32 (cho GPU Ampere+), và --sdpa-attn (cho PyTorch 2.x).

- **Tiềm năng ứng dụng:**

- **Phân tích Dữ liệu Tự động:** Hỗ trợ các nhà phân tích nghiệp vụ bằng cách tự động trích xuất dữ liệu (chart-to-table) và tóm tắt thông tin chi tiết từ một lượng lớn biểu đồ.
- **Công cụ Hỗ trợ Lập trình:** Giúp người dùng tạo mã Python (ví dụ: Matplotlib) để tái tạo hoặc tùy chỉnh các biểu đồ một cách nhanh chóng.
- **Hỗ trợ Ra quyết định:** Trả lời các câu hỏi tính toán phức tạp (ví dụ: "percentage wholesale is less than retail") bằng cách sinh và thực thi mã, cung cấp các câu trả lời chính xác dựa trên dữ liệu.
- **Giáo dục và Đào tạo:** Dùng làm công cụ dạy học, giúp sinh viên hiểu cách phân tích biểu đồ thông qua các bước giải chi tiết (Program-of-Thoughts).

CHƯƠNG 5. HẠN CHẾ VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

Mặc dù mô hình TinyChart đã đạt được những kết quả ấn tượng, vượt trội hơn các mô hình lớn hơn, công trình [1] cũng đã thẳng thắn chỉ ra các hạn chế còn tồn tại, từ đó mở ra các định hướng phát triển trong tương lai.

5.1. Hạn chế của mô hình

Dựa trên các phân tích và nghiên cứu tình huống (Case Studies) trong bài báo, mô hình TinyChart bộc lộ một số hạn chế cố hữu:

- **Hiện tượng "ảo giác" (Hallucination):** Tương tự như các MLLM khác, TinyChart vẫn có thể tạo ra các nội dung "ảo giác" (Mục 4.6, Hình 8). Trong tác vụ tóm tắt biểu đồ (Chart-to-Text), mô hình có thể mô tả các thông tin hoặc số liệu không hoàn toàn khớp với biểu đồ, ngay cả khi nó đã nhận diện đúng các giá trị. Đây là một thách thức chung của các mô hình ngôn ngữ lớn.
- **Khả năng tái tạo và nhận thức không gian hạn chế:** Hạn chế này thể hiện ở hai khía cạnh.
 - **Tái tạo biểu đồ phức tạp (Hình 9):** Mô hình thất bại trong việc vẽ lại các loại biểu đồ phức tạp và chưa từng thấy trong dữ liệu huấn luyện, đặc biệt là biểu đồ 3D. Điều này cho thấy khả năng tổng quát hóa về mặt cấu trúc không gian 3D của mô hình còn yếu.
 - **Ước lượng giá trị (Hình 7):** Khi trích xuất dữ liệu (Chart-to-Table), nếu một điểm dữ liệu không có nhãn OCR rõ ràng, mô hình gặp khó khăn trong việc *ước lượng* giá trị chính xác của nó dựa trên vị trí tương đối với trục tọa độ. Điều này cho thấy sự yếu kém trong việc hiểu sâu về mối quan hệ không gian-tọa độ.

5.2. Định hướng phát triển

Từ chính các hạn chế nêu trên, bài báo đã trực tiếp và gián tiếp đề xuất các định hướng phát triển trong tương lai:

- **Tối ưu hóa chiến lược kết hợp PoT và Trả lời trực tiếp:** Bài báo chỉ ra rằng hiệu suất của mô hình khi ở chế độ "Oracle" (luôn chọn được câu trả lời đúng giữa PoT và trả lời trực tiếp) cao hơn đáng kể so với chế độ "Combine" (kết hợp bằng quy tắc đơn giản). Cụ thể, trong Bảng 3, TinyChart@768 đạt 89.12% ở "Oracle" nhưng chỉ 83.60% ở "Combine". Điều này cho thấy một khoảng trống lớn để cải thiện. Một hướng phát triển quan trọng là nghiên cứu một cơ chế "bộ định tuyến" (router) thông minh hơn, có khả năng tự động quyết định khi nào nên sử dụng PoT (cho câu hỏi tính toán) và khi nào nên trả lời trực tiếp (cho câu hỏi đọc-hiểu) để đạt được hiệu suất gần với mức "Oracle".
- **Mở rộng và đa dạng hóa dữ liệu huấn luyện:** Hạn chế về việc không thể tái tạo biểu đồ 3D (Hình 9) trực tiếp gợi ý rằng mô hình cần được tiếp xúc với nhiều loại biểu đồ đa dạng hơn. Bài báo đề xuất hướng giải quyết là "cải thiện phạm vi bao phủ của các loại biểu đồ khác nhau trong dữ liệu huấn luyện thông qua các kỹ thuật xây dựng dữ liệu tự động". Việc mở rộng bộ dữ liệu huấn luyện, đặc biệt là các loại biểu đồ phức tạp và hiếm gặp, là một hướng đi cần thiết để tăng cường khả năng tổng quát hóa của mô hình.
- **Nghiên cứu các phương pháp giảm thiểu "ảo giác":** Vấn đề "ảo giác" (Hình 8) là một hạn chế cố hữu. Bài báo có trích dẫn các công trình liên quan [17, 18, 22, 56] như một gợi ý rằng việc áp dụng các kỹ thuật giảm thiểu ảo giác chuyên biệt cho MLLM có thể cải thiện độ tin cậy của TinyChart trong các tác vụ mô tả và tóm tắt.

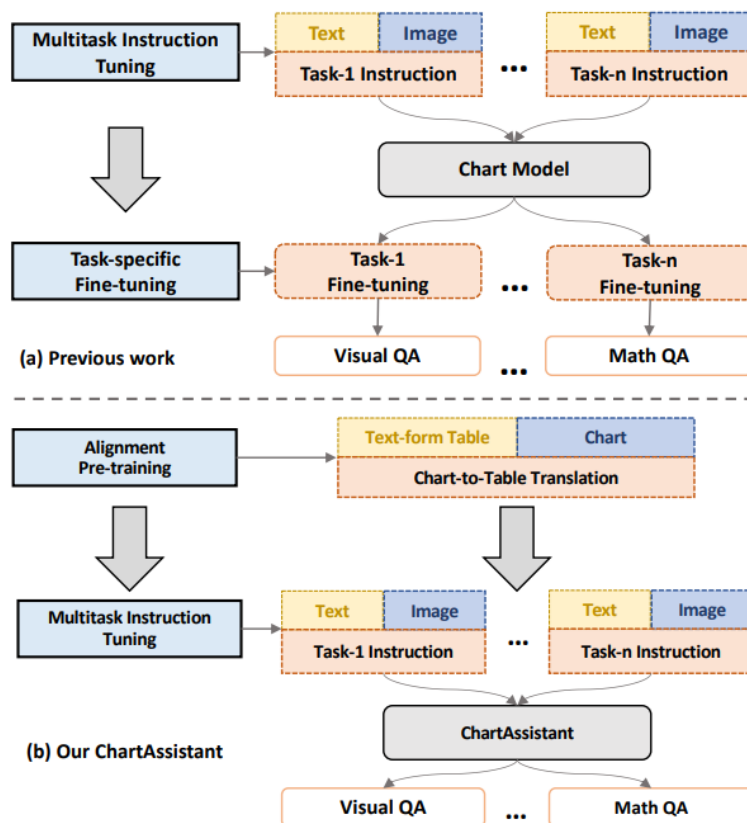
CHƯƠNG 6. PHÂN TÍCH MỞ RỘNG: ChartAssistant

ChartAssistant (ChartAst) là một mô hình ngôn ngữ lớn đa phương thức đương đại, được thiết kế chuyên biệt cho bài toán hiểu biểu đồ. Tại thời điểm TinyChart ra mắt, ChartAst là một trong những mô hình mã nguồn mở 13B mạnh nhất, đóng vai trò là đối thủ cạnh tranh (SOTA baseline) trực tiếp.

Cả hai mô hình đều nhận diện chính xác vấn đề cốt lõi: các MLLM truyền thống **thường xuyên mắc lỗi khi thực hiện các phép tính số học**. Tuy nhiên, hai mô hình đề xuất hai giải pháp kỹ thuật hoàn toàn khác nhau để giải quyết thách thức chung này.

6.1. Kiến trúc mô hình

ChartAssistant đề xuất một quy trình huấn luyện 2 giai đoạn:



Hình 10: So sánh giữa các mô hình dựa trên biểu đồ trước đây và ChartAssistant được đề xuất. ChartAssistant trước tiên sẽ căn chỉnh biểu đồ và văn bản bằng cách huấn luyện trước tác vụ dịch biểu đồ sang bảng.

- **Giai đoạn I: Tiền huấn luyện Căn chỉnh (Alignment Pre-training):** Mô hình được tiền huấn luyện với một tác vụ cơ bản là **Chart-to-Table Translation** (Dịch từ Biểu đồ sang Bảng) . Giai đoạn này giúp mô hình học cách "đọc" các yếu tố thị giác của biểu đồ và căn chỉnh chúng với cấu trúc dữ liệu dạng bảng (văn bản) . Độ mất mát trong huấn luyện trong giai đoạn II được tính bằng:

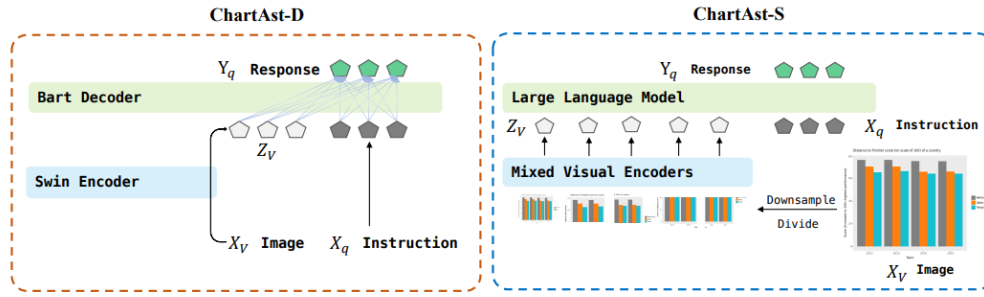
$$\mathcal{L}^{stage1} = \sum_{i=1} \log P_{\theta}(Y_{q,i}^{c2t} | X_v^{c2t}, X_q^{c2t}, Y_{q,<i}^{c2t})$$

- **Giai đoạn II: Tinh chỉnh Hướng dẫn Đa nhiệm (Multitask Instruction Tuning):** Sau khi đã căn chỉnh, mô hình được tinh chỉnh trên một tập dữ liệu hướng dẫn (instruction) đa dạng có tên là **ChartSFT**. Bộ dữ liệu này bao gồm 5 tác vụ chính: Chart-to-Table, Tóm tắt (Summarization), Hỏi đáp Mở (Open-ended QA), Hỏi đáp Tham chiếu (Referring QA), và Hỏi đáp Số học (Numerical QA). Độ mất mát trong huấn luyện trong giai đoạn II được tính bằng:

$$\mathcal{L}^{stage2} = - \sum_{k \in \Omega} \sum_{i=1} \log P_{\theta}(Y_{q,i}^k | X_v^k, X_q^k, Y_{q,<i}^k)$$

ChartAssistant có hai phiên bản kiến trúc:

- **ChartAst-D (260M):** Xây dựng dựa trên **Donut**, sử dụng Swin Encoder và BART Decoder .
- **ChartAst-S (13B):** Phiên bản SOTA, xây dựng dựa trên **SPHINX** [23], sử dụng "Mixed Visual Encoders" (bộ mã hóa thị giác hỗn hợp) và một LLM 13B .



Hình 11: Kiến trúc của ChartAst-D và ChartAst-S

6.2. Cơ chế hoạt động: Suy luận Số học bằng JSON-COT

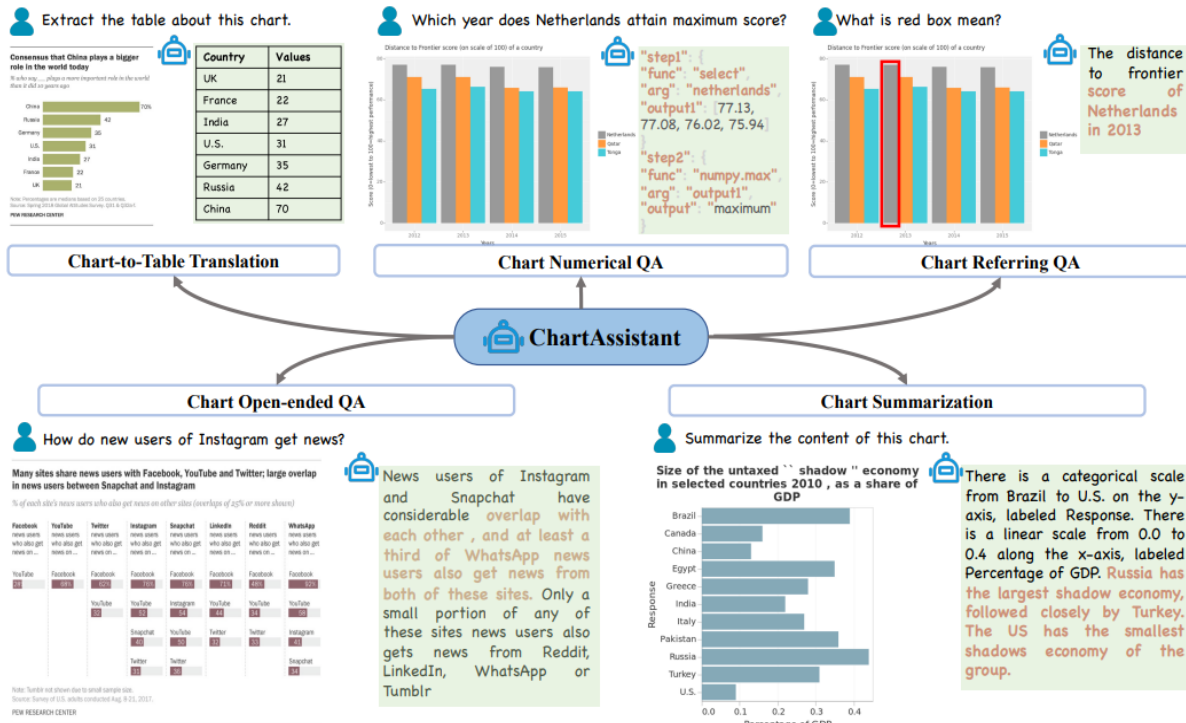
Đây là điểm khác biệt mấu chốt so với TinyChart. Để giải quyết lỗi tính toán, ChartAssistant **không** sinh mã Python. Thay vào đó, nó giới thiệu một phương pháp suy luận theo bước bằng cách:

- **Tạo Chú thích Chain-of-Thought (COT):** Mô hình được huấn luyện để sinh ra các chú thích COT, giúp phân rã bài toán phức tạp thành các bước nhỏ.
- **Định dạng JSON có thể thực thi:** Quan trọng nhất, các bước suy luận này được cấu trúc dưới dạng **lệnh JSON (executable JSON-formatted text)**.
- **Backend thực thi:** Một hệ thống backend (bộ xử lý tính toán) bên ngoài sẽ đọc các lệnh JSON này và thực thi chúng để đảm bảo kết quả số học chính xác.

Ví dụ về cơ chế này được thể hiện rõ trong Hình 2 và Hình 10 của bài báo ChartAssistant. Để trả lời câu hỏi "Năm nào Hà Lan đạt điểm tối đa?", mô hình sẽ sinh ra chuỗi lệnh JSON :

- step1: {"func": "select", "arg": "netherlands", "output": "77.13, 77.08, ..."} (Chọn dữ liệu)

- step2: {"func": "numpy.max", "arg": "output1", "output": "maximum"} (Tìm giá trị lớn nhất)



Hình 12: ChartAssistant được huấn luyện sẵn về nhiều nhiệm vụ liên quan đến biểu đồ, và có thể thực hiện thành thạo nhiều nhiệm vụ hiểu biểu đồ và lập luận, bao gồm chuyển đổi biểu đồ sang bảng, kiểm tra chất lượng số, kiểm tra chất lượng tham

6.3. So sánh 2 mô hình (ChartAssistant vs. TinyChart)

Bảng dưới đây tóm tắt các khác biệt cơ bản giữa hai mô hình, sử dụng dữ liệu hiệu suất được báo cáo trong bài báo TinyChart (mô hình đã đánh bại ChartAssistant).

Yếu tố	ChartAssistant (ChartAst-S)	TinyChart (Mô hình mở rộng)
Kích thước Mô hình	13 tỷ tham số	3 tỷ tham số
Độ phân giải Ảnh	448x448	768x768 (Hiệu quả hơn nhờ VTM)
Suy luận Số học	JSON-COT dựa trên mẫu	Program-of-Thoughts (PoT) (Sinh mã Python)

Hiệu suất (ChartQA Avg.)	79.90%	83.60%
Hiệu suất (ChartQA Human)	65.90%	73.34%
Tốc độ Suy luận	1.47 it/s	3.14 it/s (Nhanh hơn 2.14 lần)

Bảng 6: Sự khác biệt giữa hai mô hình ChartAssistant vs. TinyChart

6.4. Kết luận mở rộng

Việc so sánh TinyChart và ChartAssistant cho thấy một bước tiến rõ rệt trong lĩnh vực hiểu biểu đồ. ChartAssistant đã đặt nền móng quan trọng bằng cách giới thiệu quy trình huấn luyện 2 giai đoạn và là mô hình tiên phong áp dụng phương pháp suy luận theo bước (JSON-COT) cho các bài toán số học .

Tuy nhiên, TinyChart đã cải tiến vượt bậc trên nền tảng này:

- **Về Suy luận (PoT > JSON-COT):** Giải pháp của ChartAst (lệnh JSON) bị chính bài báo TinyChart chỉ trích là *không linh hoạt* và có *phạm vi hẹp* (chỉ bao phủ các kịch bản có sẵn trong mẫu) . Ngược lại, **PoT (sinh mã Python)** của TinyChart tổng quát hơn rất nhiều , cho phép mô hình tự "lập trình" các bước suy luận phức tạp, đa bước (ví dụ: `Answer=np.mean(np.sort(Y)[:2])`), điều mà hệ thống JSON của ChartAst không thể làm được.
- **Về Hiệu suất (VTM):** Điều đáng kinh ngạc là TinyChart (3B) không chỉ nhanh hơn mà còn chính xác hơn ChartAst (**13B**) . Điều này đạt được là nhờ **Visual Token Merging (VTM)**. VTM cho phép TinyChart sử dụng ảnh đầu vào có độ phân giải *cao hơn* (768x768) so với Ast (448x448), giúp nó "nhìn" rõ các chi tiết OCR hơn . Trong khi đó, nó vẫn giữ được chuỗi token thị giác ngắn (732 token), giúp đạt tốc độ suy luận nhanh gấp đôi đối thủ .

Tóm lại, TinyChart không chỉ đơn thuần là một mô hình nhỏ hơn, mà nó còn đại diện cho một bước tiến về chất lượng kiến trúc, sử dụng các phương pháp (VTM và PoT) thông minh hơn để vượt qua các đối thủ 13B lớn hơn cả về hiệu suất lẫn tốc độ.

CHƯƠNG 7. KẾT LUẬN

7.1. Kết luận dự án

Qua quá trình phân tích chi tiết, bài luận đã làm rõ những đóng góp và hiệu quả của mô hình TinyChart trong lĩnh vực hiểu biểu đồ tự động. Công trình này đã giải quyết thành công hai thách thức cốt lõi mà các Mô hình Ngôn ngữ lớn Đa phương thức (MLLM) truyền thống gặp phải, vốn đã được thảo luận trong phần kiến thức nền tảng (Chương 2).

TinyChart đã chứng minh một cách thuyết phục rằng các mô hình MLLM nhỏ, chuyên biệt có thể đạt được hiệu suất vượt trội so với các mô hình lớn hơn nhiều thông qua các cải tiến kiến trúc và chiến lược huấn luyện thông minh.

Hiệu quả này được thể hiện rõ qua hai đóng góp kỹ thuật chính. Thứ nhất, mô hình đã giải quyết vấn đề chi phí tính toán với ảnh độ phân giải cao bằng cách tích hợp mô-đun **Gộp Token Thị giác (Visual Token Merging - VTM)**. Phương pháp này cho phép xử lý hiệu quả hình ảnh ở độ phân giải cao (ví dụ: 768x768) bằng cách giảm đáng kể độ dài chuỗi đặc trưng thị giác, qua đó tránh được lỗi hết bộ nhớ (OOM) mà vẫn giữ được các chi tiết OCR quan trọng.

Thứ hai, TinyChart giải quyết vấn đề suy luận số học yếu bằng cách áp dụng chiến lược học **Program-of-Thoughts (PoT)**, một phương pháp đã "tách biệt" thành công khả năng suy luận logic khỏi việc thực thi tính toán. Bằng cách huấn luyện mô hình tạo ra các đoạn mã Python và giao phó việc tính toán cho một trình thông dịch, độ chính xác trên các câu hỏi tính toán đã được cải thiện đáng kể.

Tóm lại, TinyChart không chỉ là một mô hình 3 tỷ tham số đạt hiệu suất SOTA, vượt qua cả các mô hình 13 tỷ tham số và GPT-4V trên benchmark ChartQA. Quan trọng hơn, nó cung cấp một phương pháp luận và một kiến trúc kiểu mẫu (blueprint) cho thấy hướng đi phát triển các MLLM hiệu quả, chuyên biệt, phá vỡ quan niệm rằng hiệu suất cao luôn đòi hỏi quy mô tham số khổng lồ.

7.2. Liên hệ sinh viên

7.2.1. Sinh viên Chu Anh Trường

- Từ những thành công thực nghiệm đầy ấn tượng của dự án TinyChart, đặc biệt là minh chứng cho việc kết hợp hiệu quả giữa kiến trúc mô hình nhỏ gọn và chiến lược suy luận qua mã lệnh, em nhận thấy một hướng nghiên cứu đầy tiềm năng là phát triển các hệ thống Phân tích dữ liệu tự động (Automated Data Analytics) dựa trên các Mô hình ngôn ngữ nhỏ (SLMs) chuyên biệt. Hướng đi này không chỉ đơn thuần là sự kế thừa tư duy "hiệu quả hơn kích thước" mà TinyChart đã khởi xướng, mà còn đề xuất một sự chuyển dịch chiến lược quan trọng trong lĩnh vực AI: thay vì chạy đua với các mô hình đa dụng khổng lồ tốn kém tài nguyên, chúng ta tập trung vào các mô hình nhỏ được tinh chỉnh sâu (deeply fine-tuned) cho từng miền nghiệp vụ cụ thể.
- Trong bối cảnh dữ liệu được ví như tài sản vô giá của kỷ nguyên số, việc ứng dụng các mô hình ngôn ngữ lớn (LLMs) vào doanh nghiệp đang gặp rào cản lớn về an toàn thông tin. Hướng nghiên cứu này giải quyết trực diện thách thức đó thông qua việc triển khai cục bộ (on-premise) các mô hình nhỏ (tương tự quy mô 3 tỷ tham số của TinyChart). Việc này giúp các tổ chức thiết lập một "bức tường lửa" bảo vệ tuyệt đối các dữ liệu nhạy cảm như báo cáo tài chính nội bộ hay thông tin định danh khách hàng, đảm bảo chúng không bao giờ phải rời khỏi hạ tầng của doanh nghiệp. Đồng thời, mô hình nhỏ giúp tối ưu hóa đáng kể bài toán chi phí vận hành và tiêu thụ năng lượng, giúp công nghệ AI tiên tiến có thể trở nên phổ thông và khả thi hơn trong các hoạt động kinh doanh hằng ngày.
- Về mặt phương pháp luận, trọng tâm cốt lõi của nghiên cứu là ứng dụng và mở rộng tư duy Program-of-Thoughts (PoT) để khắc phục điểm yếu chí mạng của các mô hình ngôn ngữ truyền thống: sự "ảo giác" (hallucination) trong tính toán số học. Các mô hình ngôn ngữ vốn hoạt động dựa trên xác suất, do đó thường xuyên sai sót khi thực hiện các phép tính trên dữ liệu bảng. Thay vì ép buộc mô hình phải "tính nhẩm", hệ thống đề xuất sẽ được huấn luyện để đóng vai trò là một lập trình viên: nó sinh ra các đoạn mã lập trình chính xác (như Python/SQL) để xử lý dữ liệu. Bằng cách kết hợp khả năng hiểu ngữ nghĩa tự nhiên mềm dẻo của SLM với sức mạnh tính toán tất định, chính xác tuyệt đối của máy tính, chúng ta loại bỏ hoàn toàn sai số trong phân tích.

- Đích đến cuối cùng của hướng nghiên cứu này không chỉ dừng lại ở việc tạo ra các công cụ hỏi đáp thụ động, mà hướng tới việc kiến tạo nên các "Tác nhân phân tích dữ liệu" (Data Analysis Agents) tin cậy. Những tác nhân này sẽ hoạt động như những trợ lý ảo thông minh, có khả năng tự động trích xuất, làm sạch, phân tích và trực quan hóa dữ liệu. Mục tiêu là cung cấp những thông tin chi tiết (insights) xác thực, hỗ trợ con người ra quyết định nhanh chóng, chính xác và an toàn hơn, qua đó hiện thực hóa tầm nhìn về một nền quản trị dựa trên dữ liệu (Data-Driven Decision Making) được hỗ trợ bởi AI.

7.2.2. Sinh viên Phạm Quang Vinh

- Trong kỷ nguyên chuyển đổi số, vai trò của Chuyên viên Phân tích Kinh doanh (**Business Analyst - BA**) đang có sự chuyển dịch mạnh mẽ, từ việc thu thập yêu cầu đơn thuần sang phân tích dữ liệu chuyên sâu để hỗ trợ ra quyết định (Data-Driven Decision Making). Tuy nhiên, một thách thức lớn mà các BA hiện đại phải đối mặt là sự tồn tại của nguồn "dữ liệu tối" (dark data) – những thông tin giá trị bị "khóa" trong các biểu đồ hình ảnh thuộc báo cáo PDF, bài thuyết trình, hay tài liệu của đối thủ cạnh tranh mà các công cụ truyền thống không thể khai thác. Dựa trên bối cảnh đó, nghiên cứu về mô hình TinyChart không chỉ là một giải pháp kỹ thuật mà còn là nền tảng chiến lược gắn liền với định hướng nghề nghiệp tương lai của em trong lĩnh vực Phân tích Kinh doanh.
- TinyChart, với khả năng "Hiểu biểu đồ hiệu quả" (**Efficient Chart Understanding**), đóng vai trò như một cầu nối thông minh giúp chuyển đổi dữ liệu phi cấu trúc thành thông tin hữu ích. Cụ thể, mô hình giải quyết ba bài toán cốt lõi của nghề BA: Thứ nhất, trong phân tích đối thủ cạnh tranh, tính năng **Chart-to-Table** cho phép tự động trích xuất dữ liệu nền từ ảnh biểu đồ, loại bỏ quy trình nhập liệu thủ công dễ sai sót. Thứ hai, để hỗ trợ ra quyết định chính xác, TinyChart ứng dụng cơ chế **Program-of-Thoughts (PoT)**. Thay vì đưa ra các con số ước lượng cảm tính, mô hình sinh mã Python để thực hiện các phép tính số học (tăng trưởng, chênh lệch, trung bình) với độ chính xác cao, đảm bảo tính tin cậy cho các khuyến nghị chiến lược. Thứ ba, trong khâu báo cáo, khả năng tóm tắt (**Summarization**) và vẽ lại biểu đồ (**Redrawing**) giúp chuẩn hóa và trực quan hóa thông tin nhanh chóng, tối ưu hóa hiệu suất làm việc của nhân sự.
- Điểm ưu việt của TinyChart nằm ở sự cân bằng giữa hiệu suất và chi phí, làm cho nó trở nên đặc biệt khả thi đối với các dự án cá nhân hoặc doanh nghiệp vừa và nhỏ (SMEs). Về

mật kỹ thuật, với kích thước nhỏ gọn 3 tỷ tham số (3B), mô hình vận hành mượt mà trên phần cứng phổ thông (như GPU RTX 3050) mà không cần hệ thống máy chủ đắt đỏ. Về mặt kinh tế và bảo mật, nhờ mã nguồn mở và khả năng triển khai cục bộ (Local Deployment), TinyChart giúp loại bỏ chi phí bản quyền API, đồng thời đảm bảo dữ liệu nhạy cảm của doanh nghiệp không bao giờ bị rò rỉ ra bên ngoài – yếu tố sống còn trong quản trị thông tin. Dù vẫn còn hạn chế nhất định với biểu đồ 3D hay hiện tượng ảo giác, nhưng với cơ chế **Visual Token Merging (VTM)** xử lý được ảnh độ phân giải cao, TinyChart vẫn chứng minh được sự vượt trội so với các giải pháp hiện hành.

- Dựa trên nền tảng đã xây dựng, định hướng nghiên cứu tiếp theo của em là đưa TinyChart vượt ra khỏi khuôn khổ một mô hình đơn lẻ để trở thành một phần của quy trình nghiệp vụ BA tự động hóa. Kế hoạch bao gồm việc xây dựng một luồng xử lý **"ETL thông minh"**, tự động quét và trích xuất dữ liệu từ hàng loạt báo cáo thị trường để nạp vào kho dữ liệu. Đồng thời, tập trung tinh chỉnh (Fine-tuning) mô hình với các dữ liệu đặc thù ngành Tài chính - Ngân hàng (như biểu đồ Nến, Waterfall) và phát triển cơ chế **"Kiểm chứng kép" (Dual-Verification)** để triệt tiêu hiện tượng ảo giác, đảm bảo tính toàn vẹn dữ liệu. Đích đến cuối cùng là tích hợp TinyChart trở thành một **"Copilot"** thông minh ngay trong các công cụ BI quen thuộc như Excel hay PowerBI, giúp người làm phân tích có thể thao tác với dữ liệu hình ảnh dễ dàng như dữ liệu văn bản.
- Tổng kết lại, TinyChart là giải pháp toàn diện giải quyết "điểm mù" về dữ liệu hình ảnh trong phân tích kinh doanh, đảm bảo độ chính xác trong tính toán tài chính và khả thi về mặt triển khai. Nghiên cứu này không chỉ dừng lại ở khía cạnh công nghệ mà còn là bước đệm vững chắc để tôi hiện thực hóa mục tiêu trở thành một chuyên gia Phân tích Kinh doanh dựa trên AI (**AI-driven Business Intelligence**), người có khả năng làm chủ công nghệ để kiến tạo lợi thế cạnh tranh từ dữ liệu.

CHƯƠNG 8: TÀI LIỆU THAM KHẢO

1. Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Versatile Vision Language Model for Understanding, Localization, Text Reading, and Beyond. arXiv:2308.12966 [cs.CV]
2. Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. 2023. Token Merging: Your ViT But Faster. In The Eleventh International Conference on Learning Representations.
<https://openreview.net/forum?id=JroZRarw7Eu>
3. Jinyue Chen, Lingyu Kong, Haoran Wei, Chenglong Liu, Zheng Ge, Liang Zhao, Jianjian Sun, Chunrui Han, and Xiangyu Zhang. 2024. OneChart: Purify the Chart Structural Extraction via One Auxiliary Token. arXiv:2404.09987 [cs.CV]
4. Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. Transactions on Machine Learning Research (2023).
5. Xiaoyi Dong, Pan Zhang, Yuhang Zang, Yuhang Cao, Bin Wang, Linke Ouyang, Xilin Wei, Songyang Zhang, Haodong Duan, Maosong Cao, et al. 2024. InternLM-XComposer2: Mastering free-form text-image composition and comprehension in vision-language large model. arXiv preprint arXiv:2401.16420 (2024).
6. Xiaoyi Dong, Pan Zhang, Yuhang Zang, Yuhang Cao, Bin Wang, Linke Ouyang, Songyang Zhang, Haodong Duan, Wenwei Zhang, Yining Li, et al. 2024. InternLM-XComposer2-4KHD: A Pioneering Large Vision-Language Model Handling Resolutions from 336 Pixels to 4K HD. arXiv preprint arXiv:2404.06512 (2024).
7. Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. CoRR abs/2010.11929 (2020). arXiv:2010.11929 <https://arxiv.org/abs/2010.11929>
8. Hao Feng, Qi Liu, Hao Liu, Wengang Zhou, Houqiang Li, and Can Huang. 2023. DocPedia: Unleashing the Power of Large Multimodal Model in the Frequency Domain for Versatile Document Understanding. arXiv preprint arXiv:2311.11810 (2023).

9. Jiayun Fu, Bin B Zhu, Haidong Zhang, Yayi Zou, Song Ge, Weiwei Cui, Yun Wang, Dongmei Zhang, Xiaojing Ma, and Hai Jin. 2022. Chartstamp: Robust chart embedding for real-world applications. In Proceedings of the 30th ACM International Conference on Multimedia. 2786–2795.
10. Yucheng Han, Chi Zhang, Xin Chen, Xu Yang, Zhibin Wang, Gang Yu, Bin Fu, and Han Wang Zhang. 2023. Chartllama: A multimodal llm for chart understanding and generation. arXiv preprint arXiv:2311.16483 (2023).
11. Dan Hendrycks and Kevin Gimpel. 2016. Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units. CoRR abs/1606.08415 (2016). arXiv:1606.08415 <http://arxiv.org/abs/1606.08415>
12. Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2023. Cogagent: A visual language model for gui agents. arXiv preprint arXiv:2312.08914 (2023).
13. Anwen Hu, Shizhe Chen, and Qin Jin. 2021. Question-controlled Text-aware ImageCaptioning. In Proceedings of the 29th ACM International Conference on Multimedia (Virtual Event, China) (MM ’21). Association for Computing Machinery, New York, NY, USA, 3097–3105. <https://doi.org/10.1145/3474085.3475452>
14. Anwen Hu, Yaya Shi, Haiyang Xu, Jiabo Ye, Qinghao Ye, Ming Yan, Chenliang Li, Qi Qian, Ji Zhang, and Fei Huang. 2024. mPLUG-PaperOwl: Scientific Diagram Analysis with the Multimodal Large Language Model. arXiv:2311.18248 [cs.MM]
15. Anwen Hu, Haiyang Xu, Jiabo Ye, Ming Yan, Liang Zhang, Bo Zhang, Chen Li, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. 2024. mPLUG-DocOwl 1.5: Unified Structure Learning for OCR-free Document Understanding. arXiv:2403.12895 [cs.CV]
16. Kung-Hsiang Huang, Hou Pong Chan, Yi R. Fung, Haoyi Qiu, Mingyang Zhou, Shafiq Joty, Shih-Fu Chang, and Heng Ji. 2024. From Pixels to Insights: A Survey on Automatic Chart Understanding in the Era of Large Foundation Models. arXiv:2403.12027 [cs.CL]
17. Qidong Huang, Xiaoyi Dong, Pan Zhang, Bin Wang, Conghui He, Jiaqi Wang, Dahua Lin, Weiming Zhang, and Nenghai Yu. 2024. OPERA: Alleviating Hallucination in Multimodal Large Language Models via Over-Trust Penalty and Retrospection-Allocation. arXiv:2311.17911 [cs.CV]

18. Chaoya Jiang, Haiyang Xu, Mengfan Dong, Jiaxing Chen, Wei Ye, Ming Yan, Qinghao Ye, Ji Zhang, Fei Huang, and Shikun Zhang. 2024. Hallucination Augmented Contrastive Learning for Multimodal Large Language Model. arXiv:2312.06968 [cs.CV]
19. Kushal Kafle, Brian Price, Scott Cohen, and Christopher Kanan. 2018. Dvqa: Understanding data visualizations via question answering. In Proceedings of the IEEE conference on computer vision and pattern recognition. 5648–5656.
20. Shankar Kantharaj, Xuan Long Do, Rixie Tiffany Leong, Jia Qing Tan, Enamul Hoque, and Shafiq Joty. 2022. OpenCQA: Open-ended Question Answering with Charts. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 11817–11837.
<https://doi.org/10.18653/v1/2022.emnlp-main.811>
21. Shankar Kantharaj, Rixie Tiffany Leong, Xiang Lin, Ahmed Masry, Megh Thakkar, Enamul Hoque, and Shafiq Joty. 2022. Chart-to-Text: A Large-Scale Benchmark for Chart Summarization. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 4005–4023. <https://doi.org/10.18653/v1/2022.acl-long.277>
22. Sicong Leng, Hang Zhang, Guanzheng Chen, Xin Li, Shijian Lu, Chunyan Miao, and Lidong Bing. 2023. Mitigating Object Hallucinations in Large Vision Language Models through Visual Contrastive Decoding. arXiv:2311.16922[cs.CV]
23. Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks Are All You Need II: phi-1.5 technical report. arXiv:2309.05463 [cs.CL]
24. Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. 2023. Evaluating Object Hallucination in Large Vision-Language Models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 292–305.
25. Ziyi Lin, Chris Liu, Renrui Zhang, Peng Gao, Longtian Qiu, Han Xiao, Han Qiu, Chen Lin, Wenqi Shao, Keqin Chen, Jiaming Han, Siyuan Huang, Yichi Zhang, Xuming He, Hongsheng Li, and Yu Qiao. 2023. SPHINX: The Joint Mixing of Weights, Tasks, and Visual Embeddings for Multi-modal Large Language Models. arXiv:2311.07575 [cs.CV]

26. Fangyu Liu, Julian Eisenschlos, Francesco Piccinno, Syrine Krichene, Chenxi Pang, Kenton Lee, Mandar Joshi, Wenhui Chen, Nigel Collier, and Yasemin Altun. 2023. DePlot: One-shot visual language reasoning by plot-to-table translation. In Findings of the Association for Computational Linguistics: ACL 2023, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 10381–10399. <https://doi.org/10.18653/v1/2023.findings-acl.660>
27. Fangyu Liu, Francesco Piccinno, Syrine Krichene, Chenxi Pang, Kenton Lee, Mandar Joshi, Yasemin Altun, Nigel Collier, and Julian Eisenschlos. 2023. MatCha: Enhancing Visual Language Pretraining with Math Reasoning and Chart Derendering. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 12756–12770. <https://doi.org/10.18653/v1/2023.acl-long.714>
28. Fuxiao Liu, Xiaoyang Wang, Wenlin Yao, Jianshu Chen, Kaiqiang Song, Sangwoo Cho, Yaser Yacoob, and Dong Yu. 2023. Mmc: Advancing multimodal chart understanding with large-scale instruction tuning. arXiv preprint arXiv:2311.10774 (2023).
29. Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2023. Improved Baselines with Visual Instruction Tuning. arXiv:2310.03744 [cs.CV]
30. Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2024. Visual instruction tuning. Advances in neural information processing systems 36 (2024).
31. Yuliang Liu, Zhang Li, Biao Yang, Chunyuan Li, Xucheng Yin, Chenglin Liu, Lianwen Jin, and Xiang Bai. 2024. On the Hidden Mystery of OCR in Large Multimodal Models. arXiv:2305.07895 [cs.CV]
32. Ahmed Masry, Xuan Long Do, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning. In Findings of the Association for Computational Linguistics: ACL 2022. 2263–2279.
33. Ahmed Masry, Parsa Kavehzadeh, Xuan Long Do, Enamul Hoque, and Shafiq Joty. 2023. UniChart: A Universal Vision-language Pretrained Model for Chart Comprehension and Reasoning. arXiv:2305.14761 [cs.CL]

34. AhmedMasry,MehradShahmohammadi,MdRizwanParvez,EnamulHoque,and Shafiq Joty. 2024. ChartInstruct: Instruction Tuning for Chart Comprehension and Reasoning. arXiv:2403.09028 [cs.CL]
35. Fanqing Meng, Wenqi Shao, Quanfeng Lu, Peng Gao, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2024. ChartAssistant: A Universal Chart Multimodal Language Model via Chart-to-Table Pre-training and Multitask Instruction Tuning. arXiv preprint arXiv:2401.02384 (2024). arXiv:2401.02384
36. Nitesh Methani, Pritha Ganguly, Mitesh M Khapra, and Pratyush Kumar. 2020. Plotqa: Reasoning over scientific plots. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. 1527–1536.
37. Jason Obeid and Enamul Hoque. 2020. Chart-to-Text: Generating Natural Language Descriptions for Charts by Adapting the Transformer Model. CoRR abs/2010.09142 (2020). arXiv:2010.09142 <https://arxiv.org/abs/2010.09142>
38. OpenAI. 2023. GPT-3.5-Turbo. <https://platform.openai.com/docs/models/gpt-3-5-turbo>.
39. OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]
40. Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems 35 (2022), 27730–27744.
41. Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In Proceedings of the 40th annual meeting of the Association for Computational Linguistics. 311–318.
42. Raian Rahman, Rizvi Hasan, Abdullah Al Farhad, Md. Tahmid Rahman Laskar, Md. Hamjajul Ashmafee, and Abu Raihan Mostofa Kamal. 2023. ChartSumm: A Comprehensive Benchmark for Automatic Chart Summarization of Long and Short Summaries. Proceedings of the Canadian Conference on Artificial Intelligence (jun 5 2023). <https://caiac.pubpub.org/pub/ujhjycsw>.
43. Anna Rohrbach, Lisa Anne Hendricks, Kaylee Burns, Trevor Darrell, and Kate Saenko. 2018. Object Hallucination in Image Captioning. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. 4035–4045.

44. Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. 2019. Towards VQA Models That Can Read. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
45. Benny Tang, Angie Boggust, and Arvind Satyanarayan. 2023. VisText: A Benchmark for Semantically Rich Chart Captioning. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 7268–7298. <https://doi.org/10.18653/v1/2023.acl-long.401>
46. Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805 (2023).
47. Stefan van der Walt, S. Chris Colbert, and Gael Varoquaux. 2011. The NumPy Array: A Structure for Efficient Numerical Computation. Computing in Science & Engineering 13, 2 (2011), 22–30. <https://doi.org/10.1109/MCSE.2011.37>
48. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. Advances in neural information processing systems 30 (2017).
49. Junyang Wang, Yuhang Wang, Guohai Xu, Jing Zhang, Yukai Gu, Haitao Jia, Ming Yan, Ji Zhang, and Jitao Sang. 2023. An llm-free multi-dimensional benchmark for mllms hallucination evaluation. arXiv preprint arXiv:2311.07397 (2023).
50. Junyang Wang, Yiyang Zhou, Guohai Xu, Pengcheng Shi, Chenlin Zhao, Haiyang Xu, Qinghao Ye, Ming Yan, Ji Zhang, Jihua Zhu, Jitao Sang, and Haoyu Tang. 2023. Evaluation and Analysis of Hallucination in Large Vision-Language Models. arXiv:2308.15126 [cs.LG]
51. Renqiu Xia, Bo Zhang, Hancheng Ye, Xiangchao Yan, Qi Liu, Hongbin Zhou, Zijun Chen, Min Dou, Botian Shi, Junchi Yan, and Yu Qiao. 2024. ChartX & ChartVLM: A Versatile Benchmark and Foundation Model for Complicated Chart Reasoning. arXiv:2402.12185 [cs.CV]

52. Jiabo Ye, Anwen Hu, Haiyang Xu, Qinghao Ye, Ming Yan, Guohai Xu, Chenliang Li, Junfeng Tian, Qi Qian, Ji Zhang, Qin Jin, Liang He, Xin Lin, and Fei Huang. 2023. UReader: Universal OCR-free Visually-situated Language Understanding with Multimodal Large Language Model. In EMNLP (Findings). Association for Computational Linguistics, 2841–2858.
53. Qinghao Ye, Haiyang Xu, Guohai Xu, Jiabo Ye, Ming Yan, Yiyang Zhou, Junyang Wang, Anwen Hu, Pengcheng Shi, Yaya Shi, Chenliang Li, Yuanhong Xu, Hehong Chen, Junfeng Tian, Qi Qian, Ji Zhang, Fei Huang, and Jingren Zhou. 2024. mPLUG-Owl: Modularization Empowers Large Language Models with Multimodality. arXiv:2304.14178 [cs.CL]
54. Qinghao Ye, Haiyang Xu, Ming Yan, Chenlin Zhao, Junyang Wang, Xiaoshan Yang, Ji Zhang, Fei Huang, Jitao Sang, and Changsheng Xu. 2023. mPLUG-Octopus: The Versatile Assistant Empowered by A Modularized End-to-End Multimodal LLM. In Proceedings of the 31st ACM International Conference on Multimedia. 9365–9367.
55. Qinghao Ye, Haiyang Xu, Jiabo Ye, Ming Yan, Anwen Hu, Haowei Liu, Qi Qian, Ji Zhang, Fei Huang, and Jingren Zhou. 2023. mPLUG-Owl2: Revolutionizing Multi-modal Large Language Model with Modality Collaboration. arXiv:2311.04257 [cs.CL]
56. Zihao Yue, Liang Zhang, and Qin Jin. 2024. Less is More: Mitigating Multimodal Hallucination from an EOS Decision Perspective. arXiv preprint arXiv:2402.14545 (2024).
57. Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. 2023. Sigmoid loss for language image pre-training. In Proceedings of the IEEE/CVF International Conference on Computer Vision. 11975–11986.
58. Liang Zhang, Anwen Hu, Jing Zhang, Shuo Hu, and Qin Jin. 2023. MPMQA: multimodal question answering on product manuals. In Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37. 13958–13966.
59. Pan Zhang, Xiaoyi Dong, Bin Wang, Yuhang Cao, Chao Xu, Linke Ouyang, Zhiyuan Zhao, Shuangrui Ding, Songyang Zhang, Haodong Duan, Hang Yan, et al. 2023. Internlm-xcomposer: A vision-language large model for advanced text-image comprehension and composition. arXiv preprint arXiv:2309.15112 (2023).

60. PengZhang, Yunlu Xu, ZhanzhanCheng, Shiliang Pu, Jing Lu, Liang Qiao, Yi Niu, and Fei Wu. 2020. TRIE: end-to-end text reading and information extraction for document understanding. In Proceedings of the 28th ACM International Conference on Multimedia. 1413–1422.
61. Baichuan Zhou, Ying Hu, Xi Weng, Junlong Jia, Jie Luo, Xien Liu, Ji Wu, and Lei Huang. 2024. TinyLLaVA: A Framework of Small-scale Large Multimodal Models. arXiv:2402.14289 [cs.LG]